Unidade 3

Apresentação

Com o crescimento constante do volume de dados e do número de usuários em sistemas e aplicações, torna-se cada vez mais importante adotar estratégias que garantam desempenho, disponibilidade e escalabilidade no armazenamento e no processamento das informações. Nesse contexto, técnicas de distribuição de dados desempenham um papel fundamental na arquitetura de sistemas modernos.

Uma dessas técnicas é o *sharding*, ou fragmentação de dados, utilizado como forma de escalabilidade horizontal. Por meio dele, os dados são particionados e distribuídos entre diferentes servidores ou *clusters*, contribuindo para o aumento da capacidade de armazenamento, da velocidade de processamento e da tolerância a falhas. Tecnologias como Apache Cassandra e MongoDB utilizam esse conceito em suas arquiteturas.

Além da fragmentação, os sistemas distribuídos também empregam estratégias de replicação, mantendo cópias de dados, arquivos ou serviços em diferentes servidores. Essa abordagem contribui para aumentar a disponibilidade dos serviços e a confiabilidade do sistema, embora também apresente desafios relacionados à gestão e à consistência das réplicas.

Nesta Unidade de Aprendizagem, você compreenderá os principais conceitos de *sharding* e replicação, conhecerá sua aplicação em ambientes distribuídos e analisará como essas estratégias são utilizadas em bancos de dados e *frameworks* modernos.

Bons estudos.

Ao final desta Unidade de Aprendizagem, você deve apresentar os seguintes aprendizados:

- Conceituar *sharding* e explicar sua relação com a escalabilidade horizontal.
- Descrever o uso de *sharding* em *clusters* locais e distribuídos.
- Comparar o funcionamento do *sharding* com o uso de índices.
- Identificar como o *sharding* é aplicado em tecnologias como Apache Cassandra e MongoDB.
- Descrever o papel da replicação em sistemas distribuídos.
- Analisar estratégias e situações práticas de aplicação de modelos de replicação.

Infográfico: Gestão da replicação

Ao escolher a replicação de dados, ou seja, manter cópias de dados em diferentes computadores, diversas decisões precisam ser tomadas.

Por exemplo, saber quando serão enviadas mensagens para sincronizar os dados, ou que tipo de replicação se deve usar, ativa ou passiva. Para responder a questões como essas, há pesquisas sobre estratégias de replicação de dados que exemplificam características da replicação a ser usada.

Veja, no Infográfico, diferentes características e classificações das estratégias de replicação existentes.

ESTRATÉGIAS DE REPLICAÇÃO DE DADOS

A replicação de dados permite uma maior disponibilidade de serviços e tolerância a falhas. Conheça algumas características e estratégias para replicação de dados.

REPLICAÇÃO DE DADOS

PROCESSO

Ativo

Nesse tipo de replicação, as operações são **realizadas em todos os gerenciadores de réplicas.**



Passivo

Os dados são divididos em cópias primárias e secundárias. As operações de atualização são feitas apenas no servidor primário e depois replicadas aos demais.



FLUXO

Síncrono

Nesse tipo de fluxo, **todas as cópias são atualizadas em uma única transação.** O fluxo síncrono bloqueia o usuário até que todos os gerenciadores possam aplicar as atualizações.



Assíncrono

Um fluxo assíncrono é quando há **uma fila de atualizações que devem ser replicadas.** Com isso, o tempo de transação é menor, porém fica mais difícil manter uma ordem e evitar atrasos.



DIREÇÃO

Unidirecional

As operações vão **apenas em uma direção**, do servidor primário para os secundários.

Bidirecional

Nesse caso, as operações podem ir em **duas direções diferentes**. É possível ter mais um servidor primário, que pode enviar e receber atualizações.

CONEXÃO

On-line

A replicação *on-line* é aquela que **ocorre a cada operação**, ou seja, quase nenhum atraso.



Off-line

No modo *off-line*, a replicação **ocorre a cada intervalo de tempo**, ou seja, pode ser quando houver uma conexão com o *cluster*.
O modo *off-line* é muito comum em aplicações para *smartphones*, pois o usuário pode estar sem conexão com a Internet.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Conteúdo do Livro: Gestão da replicação

A replicação é parte fundamental no contexto de sistemas distribuídos. É a seção que estuda estratégias para gerenciar cópias de recursos em diferentes computadores. No entanto, existem diversos fatores que agravam a complexidade dessa tarefa, como, por exemplo, a falta de um relógio global entre os servidores replicados.

No capítulo Gestão da replicação, da obra *Sistemas distribuídos*, base teórica desta Unidade de Aprendizagem, você vai ver exemplos de estratégias e modelos para replicar objetos em diferentes servidores, além de aplicações que utilizam esses modelos para garantir maior disponibilidade a seus clientes.

Boa leitura.

SISTEMAS DISTRIBUÍDOS

Ronaldo Cesar Mengato Junior



SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS

Gestão da replicação

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Descrever os modelos de sistema e o papel da replicação.
- Examinar exemplos práticos de replicação e estratégias.
- Ilustrar aplicações de modelos de replicação.

Introdução

A replicação é fundamental nos sistemas distribuídos modernos, seja para a melhoria de desempenho, aumento da disponibilidade de serviços ou tolerância a falhas. Além da replicação de dados, é comum, atualmente, replicar rotinas que executam pequenas tarefas, chamadas de microsserviços.

Apesar de trazer algumas vantagens para a gestão, manter réplicas em diversos computadores pode ser uma estratégia bastante complexa e difícil para manter a consistência dos dados replicados. Há aplicações que precisam escolher entre alta disponibilidade ou rígida política de consistência.

Neste capítulo, você conhecerá o papel da replicação e as dificuldades inerentes a ela, além das estratégias e modelos utilizados na gestão de dados replicados em diferentes computadores. Por fim, verá alguns exemplos de aplicações reais, para exemplificar instâncias concretas dos componentes discutidos nos modelos.

1 Modelos de sistemas e função da replicação

O fato de podermos manter cópias de informações em vários computadores habilita o uso de sistemas mais complexos e robustos, o que torna a replicação bastante utilizada atualmente. Por exemplo, se você acessou a internet hoje, é provável que tenha acessado dados no cache do seu navegador, ou seja,

a informação do *site* que você buscou estava salva em uma cópia no seu próprio computador. Veja as principais vantagens de se utilizar a replicação a seguir.

- Melhoria do desempenho da aplicação: o exemplo do cache de dados pelo navegador é uma melhoria no tempo de resposta da aplicação, pois o cliente não precisa acessar o servidor remoto que possui uma latência de rede maior. Essa melhoria não se limita apenas ao cache do navegador, mas qualquer aplicação pode utilizar essa técnica.
- Maior disponibilidade: talvez um dos principais motivos para usarmos dados replicados. Atualmente, é inaceitável esperar para obter dados, por exemplo, os de redes sociais, que devem ser entregues imediatamente, ou perderão usuários. Por isso, a maioria dos serviços possuem uma taxa de disponibilidade próxima dos 100%, ou seja, a aplicação fica disponível 24 horas por dia, 7 dias por semana, o que só é possível se os dados forem replicados.
- Tolerância a falhas: alguns serviços precisam de um comportamento rigorosamente correto, mesmo que haja falhas durante a execução, o objetivo deve ser alcançado. O fato de existirem dados de alta disponibilidade não significa, necessariamente, que são corretos ou atualizados. Imagine o caso de uma rede social em que um usuário troca a sua foto de perfil, essa informação não tem necessidade de ser replicada imediatamente para todos os outros usuários; porém, isso não pode ocorrer em sistemas críticos, como um controle de tráfego aéreo, no qual os dados corretos são necessários em escalas de tempo curtas.

Apesar de ser um processo complexo, que pode envolver diversas estratégias, o fato de uma aplicação possuir dados replicados deve ser transparente ao usuário, ou seja, ele não precisa saber que existem várias cópias físicas dos dados. Além disso, em geral, os usuários esperam receber apenas um determinado conjunto de dados, mesmo que o serviço opere com mais, devendo retornar somente aquilo que foi solicitado (COULOURIS *et al*., 2013).

Outro exemplo de requisito para dados replicados é a consistência, que pode variar de acordo com o tipo da aplicação, algumas mais rígidas outras mais flexíveis. As operações de replicação em diferentes computadores devem produzir resultados que satisfaçam os requisitos da aplicação. Por exemplo, um sistema bancário que deve replicar, para os outros bancos, uma operação de crédito e outra de débito, exatamente nessa ordem, pode apresentar um valor de saldo da conta diferente e causar problemas ao usuário, se, por acaso, algum dos servidores inverterem as operações.

Segundo Tanenbaum e Steen (2008), manter várias cópias consistentes pode causar problemas de escalabilidade. Portanto, compreende-se como consistência, quando qualquer operação de consulta em qualquer réplica de dados retorna o mesmo conjunto de informações. Por isso, muitas vezes, as aplicações precisam aumentar a escalabilidade (a capacidade de crescer rapidamente) ou manter um esquema de dados consistente.

A Figura 1 mostra um modelo de arquitetura básica de gestão da replicação, definido por Coulouris *et al*. (2013), no qual as réplicas dos dados são mantidas pelos "gerenciadores de réplica", que (usando a arquitetura cliente-servidor) podem ser qualquer dispositivo que esteja servindo aos dados replicados. Portanto, podem ser chamados simplesmente de servidores.

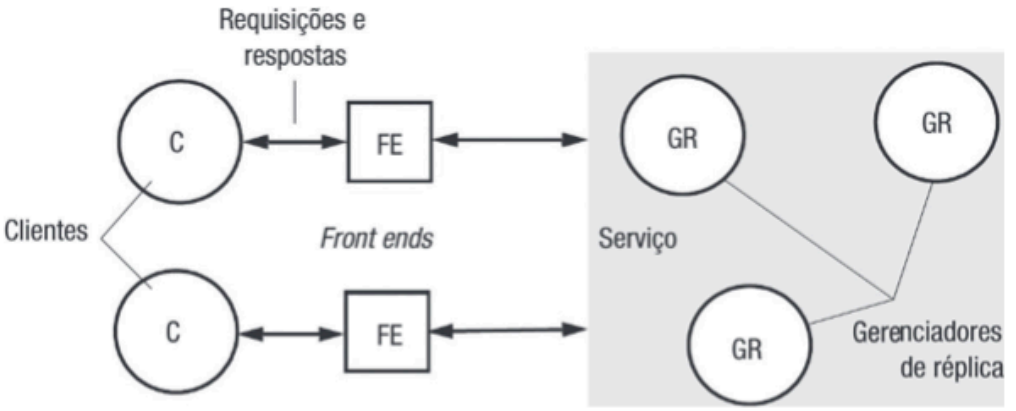


Figura 1. Modelo de arquitetura básica para gestão da replicação.

Fonte: Coulouris *et al*. (2013, p. 769).

As operações que um gerenciador executa em suas réplicas devem ser recuperáveis e atômicas (indivisíveis), ou seja, a operação não deve deixar a réplica inconsistente. Caso haja falha durante o processo, deve ser possível desfazer. Essas características, à medida que se tem mais servidores replicados, traz complexidade para a aplicação, principalmente considerando sistemas tolerantes a falhas. É preciso considerar o consenso de tempo entre as máquinas e os tempos das mensagens de sincronização.

No modelo especificado na Figura 1, os clientes fazem requisições de serviços aos servidores, e essas chamadas podem envolver operações de leitura de dados ou de atualização (quando há escrita ou alteração de um objeto ou arquivo). Antes de chegar ao serviço de destino, a requisição passa por um componente chamado *front-end*, cuja função é se comunicar com um ou mais gerenciadores de réplica, em vez de obrigar o cliente a fazer isso, tornando o processo transparente ao usuário (COULOURIS *et al*., 2013).

É importante entender o funcionamento do modelo quando uma requisição é disparada, para isso, veja a seguir as cinco fases que, segundo Coulouris *et al*. (2013), podem afetar o desempenho.

1. Requisição: é disparada pelo *front-end* para um ou mais gerenciadores. Caso seja para apenas um gerenciador, ele pode se comunicar com outros nós, contudo, o *front-end* também pode fazer um *multicast* para todos os servidores e aguardar a resposta de alguns.
2. Coordenação: ocorre quando os gerenciadores verificam se a requisição deve ser aplicada e qual a sua ordem, caso haja outras esperando. Podem ser adotados diferentes tipos de ordenação, como *first-in first-out* (FIFO), ordem causal ou ordem total.
3. Execução: a requisição executada de forma que possa ser desfeita em caso de falhas posteriores. Dependendo da implementação, a execução pode ocorrer em apenas um servidor ou em todos, caso o *front-end* tenha feito um *multicast*.
4. Acordo: nessa fase, os gerenciadores entram em um consenso sobre a requisição (se ela será aceita ou cancelada).
5. Resposta: um ou mais gerenciadores respondem ao *front-end*. Dependendo da implementação do *front-end*, ele pode pegar a primeira resposta e descartar as outras, ou aguardar a chegada de todas e, então, decidir.

Algumas aplicações podem não executar todas essas fases ou diversificar a ordem de cada uma delas. Imagine um aplicativo de *smartphone* que funcione com dados salvos localmente para quando o usuário estiver sem conexão com a internet. Nesse caso, quando surgir uma requisição, o *front-end* não poderá comunicar outros gerenciadores e deverá "pular" para a execução e uma resposta o mais rápido possível. Para manter a consistência, quando o usuário estiver conectado novamente, deverá ser replicada a requisição para outros servidores.

2 Estratégias de replicação

Nesta seção, serão abordadas duas estratégias de replicação: a passiva (*backup*), que é a mais comum, e a ativa.

No modelo de replicação passiva há a presença de um servidor primário e outros secundários (também chamados de *master* e *slaves*). A comunicação do *front-end* ocorre apenas com o gerenciador primário, em seguida, ele replica as operações realizadas para os servidores secundários. Caso o servidor primário falhe, um novo deve ser eleito como primário.

A Figura 2 exemplifica a troca de mensagens entre os componentes no modelo passivo de replicação, note que as setas saindo dos componentes de *front-end* vão apenas para um único gerenciador de réplica. Nesse cenário, a sequência de eventos quando um cliente solicita uma operação é a seguinte (COULOURIS *et al*., 2013):

- requisição — emitida pelo *front-end* para o gerenciador primário, podendo ter um identificador único para facilitar a sua coordenação;
- coordenação — o gerenciador verifica se a requisição já foi feita, devido ao fato de ela conter o seu identificador. Para que isso seja possível é necessário que o servidor mantenha um registro histórico das requisições atendidas;
- execução — a requisição é executada pelo servidor primário, e sua resposta deve ser armazenada, pois deve-se garantir a atomicidade e a capacidade de replicar essa operação;
- acordo — caso seja uma operação de atualização, o gerenciador deve emitir uma mensagem para que os outros também a executem. Os nós secundários devem enviar uma confirmação;
- resposta: o gerenciador primário envia a resposta ao *front-end* que retornará ao cliente final.

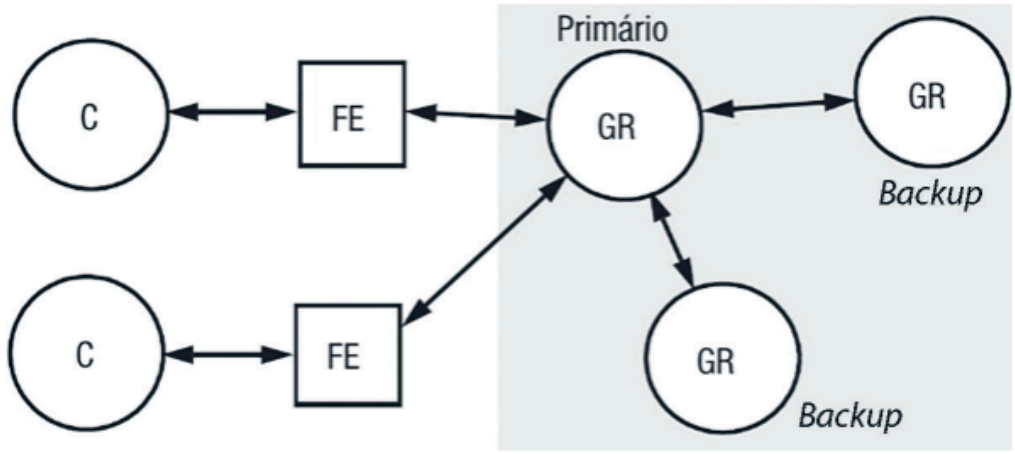


Figura 2. Modelo de replicação passiva (*backup* primário).
Fonte: Adaptada de Coulouris *et al.* (2013).

Olhando para o fluxo de operações realizado a cada requisição, é possível notar um problema que pode impactar a aplicação. Caso o sistema seja síncrono, ou seja, o cliente fique aguardando uma resposta, a etapa acordo pode causar lentidão, dependendo do número de gerenciadores secundários dentro do arranjo.

Outra preocupação com esse modelo é o caso de falha do gerenciador primário, em que um dos gerenciadores de *backup* deve assumir como primário, se mais de um assumir, causará falha no sistema. Os *backups* remanescentes devem concordar com o novo e, ainda, concordar com as operações que já foram realizadas, por isso é importante a replicação do histórico de execuções.

Para que o modelo de *backup* primário tolere até "F" falhas, é necessário F + 1 gerenciadores de réplica, o que pode ser uma implantação bastante cara, dependendo da aplicação. Além disso, esse modelo não pode tolerar falhas bizantinas. Em contrapartida, o *front-end* pode facilmente tolerar falhas, pois, uma vez que detecta uma, basta pesquisar pelo novo endereço do servidor primário.

Outra desvantagem desse modelo é a sobrecarga no gerenciador primário, pois todas as requisições chegam a ele primeiro. Uma variação que pode auxiliar esse problema é habilitar requisições de consultas, que não alteram dados e podem ser atendidas pelos gerenciadores de *backup*.

Saiba mais

Falhas bizantinas são aquelas em que os gerenciadores de réplicas não conseguem concordar com uma estratégia para falhas catastróficas no processo de replicação.

No modelo de replicação ativa, apresentado na Figura 3, os gerenciadores são organizados em grupos. O *front-end*, em vez de enviar suas requisições para apenas um servidor, envia para todos do grupo via *multicast*. Todos os nós processam a requisição de forma independente e enviam uma resposta ao *front-end*. Com isso, caso alguns deles falhem, não há impacto na operação e só haverá erro em caso de colapso geral.

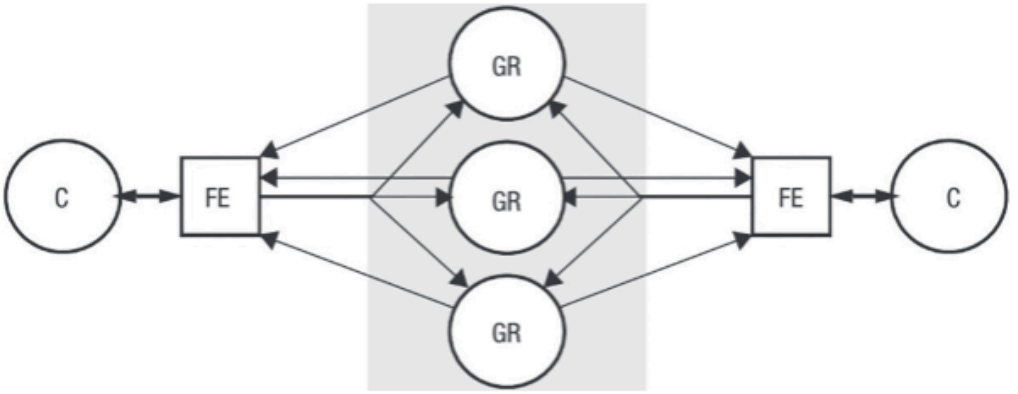


Figura 3. Modelo de replicação ativa.
Fonte: Coulouris *et al.* (2013, p. 781).

A execução de uma operação no modelo ativo, segundo Coulouris *et al.* (2013), segue as seguintes etapas:

- requisição — o *front-end* envia, via *multicast* totalmente ordenado e confiável, a requisição para todos os gerenciadores, garantindo que todos os receptores receberão a sua mensagem;
- coordenação — um sistema de comunicação em grupo envia a requisição para cada gerenciador de réplica na mesma ordem;

- execução — todos os nós executam a mesma requisição de forma similar, com isso, ao final da execução, todos devem estar no mesmo estado, garantindo a consistência;
- acordo — como a distribuição da requisição é total, não há necessidade de acordo entre os gerenciadores;
- resposta — cada gerenciador deve enviar a sua resposta ao *front-end* com o identificador da requisição; evidentemente, aqueles servidores que falharam durante a execução não enviam resposta alguma.

Considerando o modelo de resposta da replicação ativa, o conceito de falha pode ser relativo à aplicação, pois é possível definir que seja considerada falha apenas se um dos gerenciadores não responder, ou apenas se todos (ou uma boa parte) não enviarem a resposta. Com isso, a implementação do *front-end* também pode variar, sendo a política de aceitar a primeira resposta e descartar as outras adotada para um ganho de *performance*.

O sucesso do modo ativo de replicação depende bastante de um algoritmo de *multicast* totalmente ordenado e confiável para a entrega das requisições. Com isso e, ao menos, 2F + 1 de gerenciadores de réplicas, o modelo pode suportar até F falhas bizantinas. A fim de garantir de forma mais rígida a tolerância a falhas, pode-se adotar a prática de as respostas serem assinadas digitalmente pelos servidores.

3 Aplicações dos modelos de replicação

Nesta seção, serão apresentados dois exemplos de aplicações que utilizam, de algum modo, os conceitos de replicação. Com isso, será possível identificar casos reais e concretos e entender a importância do tema em sistemas distribuídos. No primeiro caso, será abordado um exemplo simples de replicação, utilizando o conceito de microsserviço, bastante popular atualmente. O segundo caso é um exemplo de implementação mais complexa da replicação em uma arquitetura desenvolvida por Ladin *et al*. (1992), que ficou conhecida pelo nome de *gossip* (fofoca).

Um microsserviço pode ser entendido como um projeto completo, porém, possui apenas uma funcionalidade. Por exemplo, um sistema que precise enviar um *e-mail* para seus clientes quando um evento ocorrer. Essa rotina pode ser escrita em um projeto a parte e ser usada em diferentes momentos na aplicação, além disso, esse serviço pode ser usado em outros projetos. Essa reusabilidade, aliada à fácil manutenção, tornou esse conceito bastante popular.

A Figura 4 apresenta um modelo para a replicação do serviço enviar *e-mails*. Perceba que, nesse caso, não há interesse em replicar dados, pois as informações necessárias para que o envio de *e-mail* ocorra podem ser passadas na própria requisição do *front-end*. Com isso, o modelo não precisa se preocupar em manter a consistência de dados, evitando uma grande complexidade.

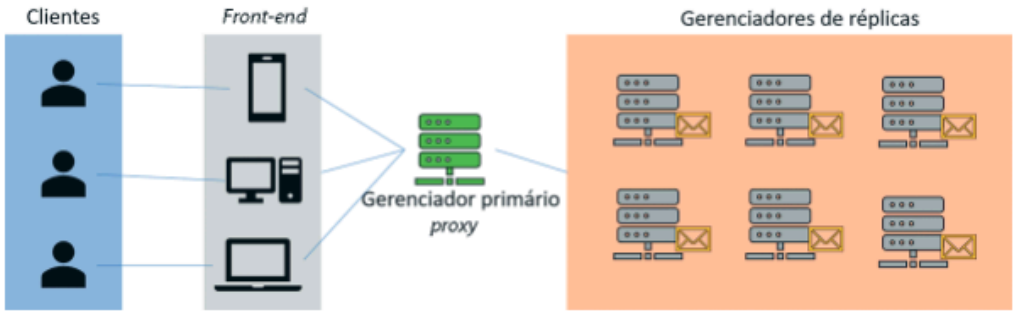


Figura 4. Modelo de replicação de serviços.

Na Figura 4 é possível reparar diferentes implementações de *front-end*. São exemplos válidos aplicativos para dispositivos móveis, *desktops*, *tablets*, *notebooks* etc. Quando um cliente deseja enviar um *e-mail*, o *front-end* manda uma requisição ao gerenciador primário, também chamado de *proxy*, que a direciona para algum gerenciador de réplica, nesse caso as instâncias do microsserviço enviar *e-mail*.

O gerenciador que atua como *proxy* possui políticas para balancear as requisições entre os servidores e nenhum fique sobrecarregado. Essa abordagem proporciona um melhor desempenho para a aplicação, pois o *proxy*, ao identificar que um servidor não retornou uma mensagem de êxito, pode atribuir a requisição para outro, e assim por diante. Se consideramos que a natureza de um envio de *e-mail* é assíncrona, não precisamos manter os clientes bloqueados enquanto as requisições são trocadas, o que traz transparência para o modelo.

O modelo de replicação de microsserviço apresentado é uma implementação do modelo passivo, mas com a exceção de não precisar manter um conjunto de dados. No caso de o *proxy* falhar, outro gerenciador pode ser eleito e exercer essa função. É importante lembrar que esses gerenciadores podem ser máquinas virtuais (ou containers), facilmente ligados ou desligados.

Outro exemplo de aplicação para a replicação é a arquitetura *gossip* explicada por Coulouris *et al*. (2013). Esse nome se refere ao fato de que os nós gerenciadores trocam mensagens de "fofoca" de tempos em tempos, a fim de transmitir as atualizações nas réplicas. Essa arquitetura é uma implementação de conceitos como replicação, consistência e sincronização. Pode ser usada por diferentes tipos de aplicações.

Os serviços da arquitetura *gossip* fornecem dois tipos de operações: consultas e atualizações, sendo que apenas a segunda modifica o estado da réplica. Os *front-ends* enviam requisições para os gerenciadores que escolherem, não necessariamente para todos ou para um primário, o que habilita a escolha de um que esteja disponível para responder em tempo razoável.

A Figura 5 apresenta as operações de consultas e atualizações em uma arquitetura *gossip*. É possível notar que, quando há uma consulta, o cliente aguarda uma resposta informando o estado da réplica, ao passo que na atualização nada é retornado. Outra característica é que cada cliente tem o seu próprio serviço, ou seja, o próprio gerenciador de réplicas ou um *cluster*. Essa característica de aplicações separa seus clientes via geolocalização, pois, assim, diminuem a latência.

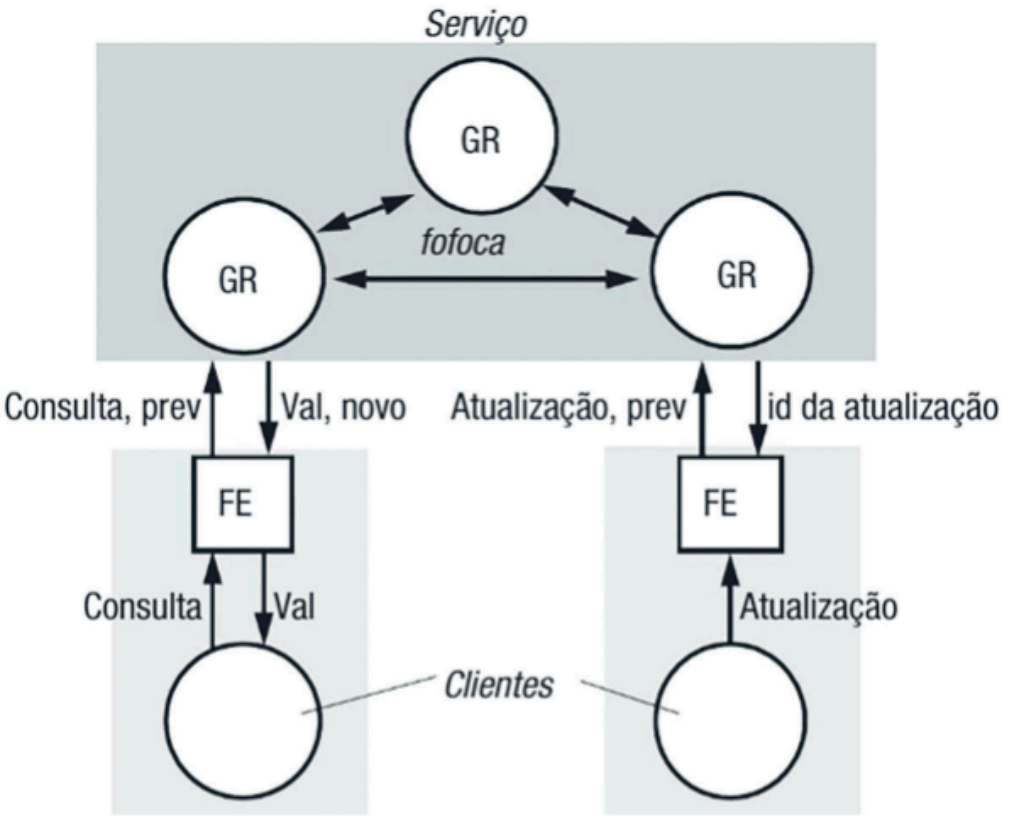


Figura 5. Operações de consulta e atualização em uma arquitetura *gossip*.
Fonte: Coulouris *et al.* (2013, p. 783).

Coulouris *et al*. (2013) explicam, de forma geral, que esse tipo de serviço processa as operações de consulta e atualização da seguinte forma:

- requisição — apesar de um *front-end* se comunicar com apenas um servidor, quando ele falha, procura outro que possa atender à requisição. Nas operações de atualização, o processo é retornado ao cliente assim que a operação tiver sido entregue ao *front-end*, então, ela é encaminhada ao servidor em segundo plano;
- resposta de atualização — o gerenciador responde assim que recebe a operação;
- coordenação — as mensagens de fofoca são a única forma de coordenação desse modelo, por isso um gerenciador não pode processar uma requisição até que possa aplicá-la, ou seja, pode ser que outros nós precisem receber atualizações antes. Esse comportamento é garantido pela ordenação total do modelo;
- execução — o gerenciador executa a requisição;

- resposta de consulta — essa fase é apenas para operações de consulta que exigem uma resposta ao cliente;
- acordo — os gerenciadores se atualizam por meio das mensagens de fofoca. Essa fase é executada de forma "preguiçosa", pois não precisa ocorrer ao término de cada requisição, e os nós são atualizados ocasionalmente, ou quando um servidor percebe que está desatualizado.

A implementação da arquitetura, em geral, fornece uma forma de consistência mais flexível, conforme você pode perceber pela sua fase de acordo. Nesse caso, considera-se que é uma consistência causal, e pode ser que algum cliente acesse dados antigos em relação a outros clientes. Outro fator que corrobora a condição de consistência relaxada é que essa arquitetura não determina um modo de propagação de atualizações, o que pode variar de acordo com a aplicação.

Uma rede social representa um cenário em que não é necessária uma consistência em tempo real e os serviços de fofoca podem ser aplicados. Por exemplo, imagine que um usuário da aplicação mora em algum país da Europa e possui amigos que também moram na mesma região e acompanham as suas publicações. O *cluster* no qual estão replicados os dados desse usuário também está localizado na Europa, assim como os dados de seus amigos. Porém, se um usuário da mesma rede social, mas que more no Brasil, tentar visualizar o perfil do usuário europeu poderá ver informações desatualizadas, pois o gerenciador de réplica daquela região não precisa enviar mensagens de fofoca com tanta frequência para os gerenciadores de outros continentes.

Essa replicação de dados com base na geolocalização ou em um chamado gráfico social é comum em aplicações distribuídas, principalmente redes sociais. As operações de consulta são mais frequentes que as de atualização, e a disponibilidade sempre vem antes da consistência. Um exemplo bastante conhecido desse tipo de organização de dados é o sistema The Associations of Objects (TAO) criado pelo Facebook (BRONSON *et al*., 2013). O *design* utilizado para a replicação no sistema TAO é o *master/slave* (primário/auxiliar), que garante que as requisições de consulta serão atendidas dentro de uma região, porém, com o risco de algumas retornarem dados obsoletos.

Os sistemas de arquivos distribuídos são aplicações que utilizam o conceito de replicação. Dois exemplos bem populares desse tipo de sistema são: Google File System (GFS) e Apache Hadoop, ambos utilizados para servir com alta disponibilidade grandes volumes de dados. O GFS utiliza o conceito de replicação passiva, mantendo cópias primárias e secundárias, ao passo

que o Hadoop pode fazer várias cópias de um dado em um mesmo servidor (GEMAYEL, 2016).

A replicação possui diversas aplicações práticas, e sua gestão possibilita alta disponibilidade, tolerância a falhas e confiabilidade, que são requisitos de muitos sistemas. Contudo, apenas usar a replicação não garante que esses requisitos sejam cumpridos, deve-se atentar ao modelo que terá o melhor desempenho na aplicação em questão. Se analisar o grande volume de dados que os sistemas computacionais devem manter atualmente, você terá certeza de que a replicação ainda será objeto de muitos estudos e experimentos.



Referências

BRONSON, N. *et al*. TAO: Facebook's Distributed Data Store for the Social Graph. *In*: USENIX ANNUAL TECHNICAL CONFERENCE, 2013, San Jose. *Proceedings* [...]. San Jose: Fairmont San Jose, 2013. p. 49–60. Disponível em: https://www.usenix.org/conference/atc13/technical-sessions/presentation/bronson. Acesso em: 24 jun. 2020.

COULOURIS, G. *et al*. *Sistemas distribuídos*: conceitos e projeto. 5. ed. Porto Alegre: Bookman, 2013. 1064 p.

GEMAYEL, N. Analyzing Google File System and Hadoop Distributed File System. *Research Journal of Information Technology*, v. 8, n. 3, p. 66–74, 2016. Disponível em: https://scialert.net/abstract/?doi=rjit.2016.66.74. Acesso em: 24 jun. 2020.

LADIN, R. *et al*. Providing high availability using lazy replication. *ACM Transactions on Computer Systems*, New York, v. 10, n. 4, p. 360–391, Nov. 1992. Disponível em: https://dl.acm.org/doi/abs/10.1145/138873.138877. Acesso em: 24 jun. 2020.

TANENBAUM, A. S.; STEEN, M. *Sistemas distribuídos*: princípios e paradigmas. 2. ed. São Paulo: Pearson Prentice Hall, 2008. 402 p.



Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.

Conteúdo:

SOLUÇÕES EDUCACIONAIS INTEGRADAS

Dica do Professor: Gestão da replicação

O compartilhamento de recursos entre usuários sempre foi um dos objetos de estudo da área de sistemas de informações. A própria gestão da replicação é uma abordagem para atingir esse objetivo. É nesse cenário que se encontram os sistemas de arquivos distribuídos, que são peças importantes em aplicações distribuídas.

Veja, nesta Dica do Professor, um exemplo de sistema de arquivos distribuídos, o CODA, e suas principais funcionalidades para o compartilhamento de arquivos.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Saiba mais: Gestão da replicação

Para ampliar o seu conhecimento a respeito desse assunto, veja abaixo as sugestões do professor:

Cluster de alta disponibilidade da informação com *software* livre

Este artigo apresenta tecnologias de *software* livre usadas para a construção de *clusters*. Algumas são para a replicação de dados e monitoramento de falhas; assim, o *cluster* pode ser capaz de alta disponibilidade e ter tolerância a falhas.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Administrando MongoDB: replicação/espelhamento

Este artigo apresenta um modelo de replicação de dados em um dos bancos de dados NoSQL mais populares, o MongoDB. É interessante conhecer como um banco não relacional lida com a replicação.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Sistemas distribuídos – abordagens de consistência e replicação

Neste vídeo, o professor Jó Uyeama fala sobre abordagens de consistência quando se trata de replicação, como consistência causal e sequencial. Além disso, apresenta estratégias de posicionamento de servidores de réplicas.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Uma introdução ao Apache Kafka, lições aprendidas em um ambiente de varejo

Este artigo traz informações sobre o Apache Kafka, uma plataforma amplamente utilizada para melhorar o desempenho de *clusters* de computadores. Uma das vantagens que essa plataforma traz é o uso de replicação para aumento da performance no fluxo de dados (*stream*). É interessante, também, ver o caso de uso mostrado pelo autor.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Infográfico: Sharding

No uso de sistemas gerenciadores de bancos de dados distribuídos, existem técnicas e arquiteturas que auxiliam no controle de requisições e consultas a dados. O uso de índices e a técnica de *sharding* (particionamento de dados) ajudam muito no momento de criar um ambiente que permita agilidade no processo de acesso a dados em servidores.

Neste Infográfico, você verá o uso de particionamento em bancos de dados, distribuídos com uso do serviço em nuvem da Google. O BigTable trabalha com o particionamento de dados em *cluster* de maneira distribuída no mundo do *big data*.

BANCOS DE DADOS PARA *BIG DATA*

Bancos de dados para *big data* têm como característica a adaptação dos processos de armazenamento e leitura de dados, uma vez que lidam com altíssimas quantidades deles.

Veja um exemplo de implantação do método *sharding* em um banco de dados não relacional (NoSQL) chamado BigTable.



BigTable

É um serviço de banco de dados para ***big data*** do **Google.** É o mesmo banco de dados NoSQL usado em vários serviços principais do Google, como Gmail, Maps, Pesquisa e Analytics.



Funcionamento

No *cloud* do BigTable, o *sharding* acontece da seguinte forma: uma tabela é fragmentada em blocos, que são conjuntos de linhas contíguas. Essa fragmentação ajuda no balanceamento da carga de trabalho das consultas.



Cluster

Com uma aplicação clusterizada, o profissional responsável e sua equipe podem projetar e determinar quantos nós (*nodes*) deverão existir, compartilhando os dados (criando partições) e até os replicando. ***Backups*** quase em tempo real podem ser criados automaticamente.



Indicação de uso

Com a escalabilidade e alta disponibilidade de dados sendo seu principal benefício, o BigTable é indicado para uso no processamento de dados não estruturados em grande volume, para criação de visualizações de relatórios e séries temporais de dados de marketing ou financeiro, para criação de gráficos, entre outros.



SGBDS que se conectam

O BigTable permite conexão direta com serviços e bancos de dados, como Cassandra, HBase, MongoDB.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Conteúdo do Livro: Sharding

Trabalhar com grandes volumes de dados requer planejamento. É importante entender bem esse fenômeno, pois a expansão do volume de dados não foi algo planejado. Como forma de compreender técnicas e ferramentas que auxiliam no processamento de grandes volumes de dados, é importante entender a importância de *frameworks* e de técnicas empregadas em sistemas gerenciadores de bancos de dados como forma de administrar toda a estrutura.

No capítulo Sharding, da obra *Framework de Big Data*, base teórica desta Unidade de Aprendizagem, você verá como os *clusters* de computadores surgem como resultado da convergência de várias tendências de computação. A disponibilidade de computadores de baixo custo, juntamente com a técnica de *sharding*, auxilia na estruturação de acesso a *softwares* por meio da computação distribuída de alto desempenho.

Boa leitura.

FRAMEWORK DE BIG DATA

Daniel dos Santos Brandão



SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS

Sharding

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Conceituar *sharding*.
- Explicar o uso de *sharding* em *clusters* locais e remotos.
- Comparar *sharding* com índices.

Introdução

Com o avanço das tecnologias de informação e comunicação, surge um grande aumento do fluxo de dados que trafegam nas redes de computadores. Lidar com dados tem obrigado instituições e empresas (públicas ou privadas e em diferentes setores da sociedade) a redobrarem seus cuidados quanto a tratamento, armazenamento e manutenção da informação contida em servidores. Por terem se tornado um dos maiores bens que uma empresa pode ter, os dados precisam estar em segurança e devem ter sua estrutura planejada para uma expansão. Um dos maiores desafios quando se trata de *big data* é a escalabilidade. Escalabilidade é a capacidade de uma empresa, ou sistema, de atender suas necessidades de crescimento sem perder qualidade, e com aumento de custos baixo (Ferreira, 2020). Também pode ser definido como habilidade de um sistema computacional tem de lidar, de forma transparente, com um número crescente de usuários ao mesmo tempo (Veras, 2016). Isso é um dos problemas mais comuns e importantes que toda empresa enfrenta, ou seja, lidar com negócios em crescimento, o que traz a necessidade de armazenamento exponencial de dados e grande demanda de disponibilidade deles.

Existem duas abordagens principais para realizar o dimensionamento do armazenamento de dados: vertical e horizontal. Independentemente da estratégia de escalabilidade que se decide seguir, é costume comprar máquinas cada vez maiores, mais rápidas e mais caras para mover os dados entre eles e realizar o aumento vertical ou agrupá-los para realizar a escala horizontalmente. Embora esse

processo funcione em grandes empresas com aporte financeiro significativo, não funciona tão bem para as pequenas e médias instituições, que precisam escalar muito além do que podem pagar. Por isso, técnicas e sistemas de código aberto ganham destaque. Nesse contexto, encaixa-se o uso do *sharding* ou fragmentação de dados. *Sharding* é uma arquitetura de escalabilidade horizontal de dados e será o foco de estudo deste capítulo.

1 O que é *sharding*?

Sharding é uma arquitetura de *big data* relativamente nova. Considerada revolucionária por profissionais da área, tem sido adotada por desenvolvedores e equipes responsáveis por projetos que lidam com dados em variados tipos de organizações. Redes sociais como Facebook, Twitter, Friendster e o Flickr já usam há algum tempo. O conceito define uma abordagem acessível para a escala horizontal dos dados sem comprometer nada. O *sharding* é, portanto, um padrão de arquitetura para sistemas distribuídos, relacionado ao particionamento horizontal, onde cada partição tem o mesmo esquema e as mesmas colunas, mas com linhas, isto é, os registros no banco em si, totalmente diferentes. Sistema distribuído é um conjunto de diferentes computadores que, interligados através de uma rede, apresentam-se ao usuário como um sistema único e com coerência (TANENBAUM; BOS, 2016). De igual modo, os dados de cada nó são exclusivos, sendo totalmente independentes entre os dados das demais partições.

Sharding é diferente da arquitetura tradicional do banco de dados de várias maneiras importantes, tendo algumas características particulares que tornam o conceito bem diferente do armazenamento de dados de bancos relacionais tradicionais. Algumas dessas características são as seguintes.

- Dados desnormalizados: em sistemas de armazenamento de dados tradicionais, os dados são normalizados, dispostos em tabelas sem anomalias e, em seguida, reunidos novamente quando precisam ser usados. Armazenam-se juntos os dados que são usados juntos, mas com a possibilidade de separar os dados por tipo. Com isso, em uma rede social, por exemplo, é possível manter os dados do perfil de um usuário separados de seus comentários, *blogs*, *e-mail*, fotos, etc. Todavia, os dados do usuário são armazenados e recuperados como um todo. Essa é uma abordagem muito rápida.

- Dados em várias instâncias físicas: historicamente, os servidores de banco de dados são ampliados aumentando o número de máquinas em um *datacenter*. Com o *sharding*, os dados são paralelos e aumentam de maneira linear. Usando essa abordagem, podem ser feitos muito mais trabalhos, porque isso acontece em processamento paralelo.
- Os dados são pequenos: quanto maior o conjunto de dados que um servidor manipula, mais difícil é obter retorno de maneira inteligente, porque há uma diversidade muito grande de dados acessados. É preciso grande quantidade de memória RAM, que pode não ser suficiente para armazenar em cache os dados quando necessário. Ao isolar os dados em fragmentos menores, é mais provável que os dados que estão sendo acessados permaneçam em cache, agilizando o processo de consulta e retorno dos dados.

Pode ser útil pensar no particionamento horizontal em termos de como ele se relaciona com o particionamento vertical. Em uma tabela particionada verticalmente, colunas inteiras são separadas e colocadas em tabelas novas e distintas. Os dados mantidos em uma partição vertical são independentes dos dados em todas as outras e cada um possui linhas e colunas distintas. O *sharding* envolve dividir os dados em dois ou mais blocos menores, chamados *shards* lógicos. Os *shards* lógicos são, então, distribuídos por nós de banco de dados separados, chamados de *shards* físicos, que podem conter vários *shards* lógicos. Apesar disso, os dados mantidos em todos os *shards* representam coletivamente um conjunto de dados lógicos inteiro.

Os fragmentos de banco de dados exemplificam uma arquitetura de dado compartilhado. Isso significa que os fragmentos são autônomos, ou seja, eles não compartilham nenhum dos mesmos dados ou recursos de computação. Em alguns casos, porém, pode fazer sentido replicar determinadas tabelas em cada *shard* para servir como tabelas de referência. Por exemplo, caso exista um banco de dados para um aplicativo que dependa de taxas de conversão fixas para medições de peso, replicar uma tabela que contenha os dados necessários da taxa de conversão em cada *shard* ajudaria a garantir que todos os dados necessários para consultas sejam mantidos em todos os *shards*. Essa é uma forma de realizar um balanço de carga.

Muitas vezes, o *sharding* é implementado no nível do aplicativo, o que significa que o aplicativo inclui código que define em qual *shard* serão permitidas leituras e gravações. No entanto, alguns sistemas de arquivos distribuídos possuem recursos de *sharding* integrados, permitindo que ele seja implementado diretamente no nível do banco de dados. O uso de apli-

cações de terceiros também é bastante utilizado, afinal, existem diferentes ferramentas que auxiliam na criação dos fragmentos de dados em bancos distribuídos.

Uma abordagem bastante recomendada para implementar *shards* de dados é a estrutura *hibernate shards*. Essa estrutura oferece *cluster* de dados críticos e suporte para particionamento horizontal, juntamente dos serviços *hibernate* padrão. Isso permite que as empresas mantenham os dados em mais de um banco de dados relacional sem nenhuma complexidade adicional ao criar os aplicativos.



Saiba mais

O *hibernate shards* é uma extensão do *hibernate core* projetada para encapsular e minimizar a complexidade de trabalhar com dados fragmentados, isto é, particionados horizontalmente. Os fragmentos de hibernação podem ser divididos conceitualmente em duas áreas, as quais você precisará entender para ter sucesso. As duas áreas são: lógica de *sharding* generalizada e lógica de *sharding* específica do aplicativo.

A **generalizada** permite que os desenvolvedores de aplicativos consultem e transacionem conjuntos de dados fragmentados usando a API principal do *hibernate core*. A **específica** garante que todo aplicativo que usa o *hibernate shards* terá suas próprias regras sobre como os dados serão distribuídos pelos seus *shards*. Em vez de tentar antecipar todas essas regras, é fornecido um conjunto de interfaces pelas quais você pode codificar a lógica de distribuição de dados do seu aplicativo (HIBERNATE SHARDS, [201–?]).

Porém, além do *hibernate*, existem outras ferramentas em que *shards* podem ser implementados por meio de sistemas de dados, como as ferramentas Apache Slice e Websphere ObjectGrid. O Slice estende o tempo de execução do OpenJPA (biblioteca em linguagem Java) para sistemas de armazenamento de dados distribuídos, possivelmente heterogêneos e particionados de maneira horizontal. A Figura 1 demonstra a utilização do *plug-in* Slice junto do OpenJPA.

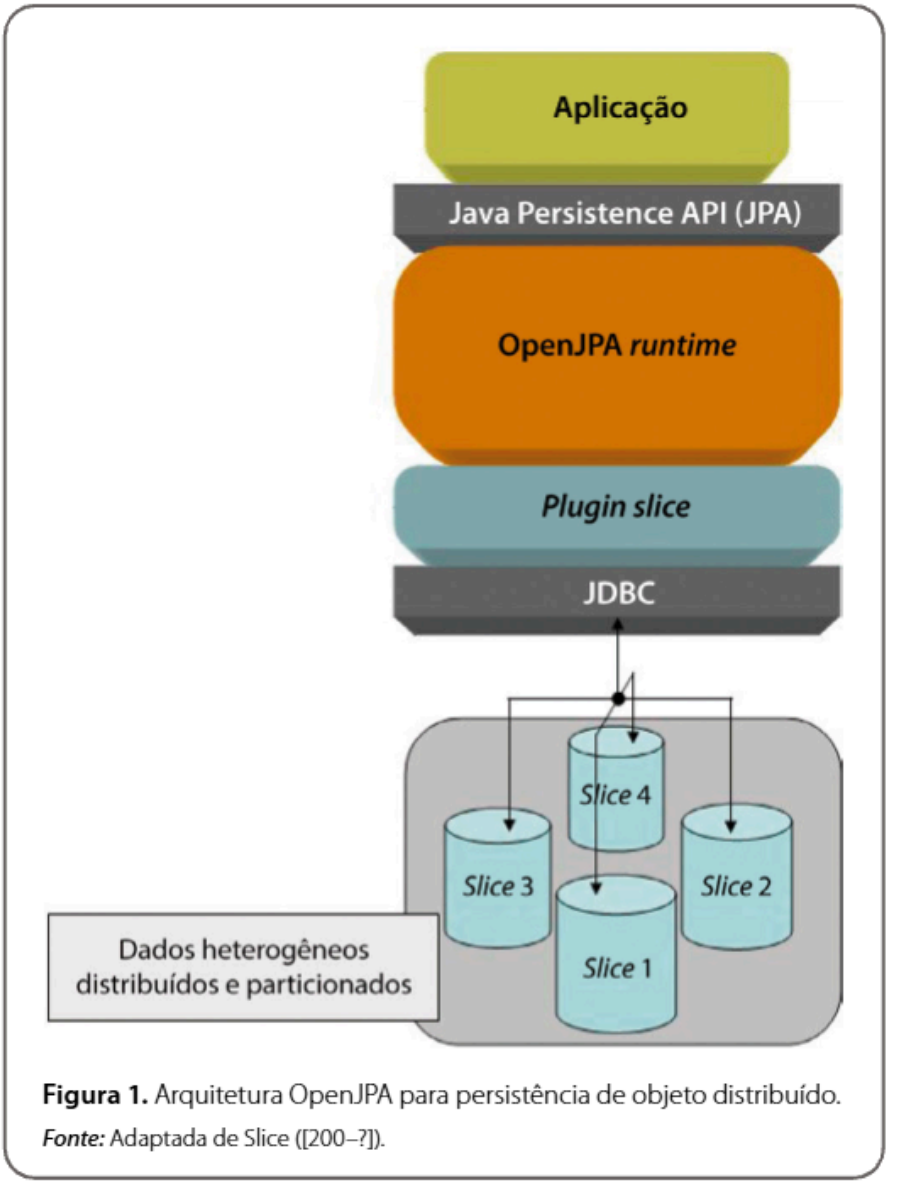


Figura 1. Arquitetura OpenJPA para persistência de objeto distribuído.
Fonte: Adaptada de Slice ([200–?]).

Portanto, o Apache Slice funciona como uma camada *plug-in* intermediando o OpenJPA e a camada JDBC como a biblioteca de conexão com bancos de dados da linguagem Java. Com o uso do *plug-in*, os dados são "fatiados" no que se chama de *slices*, realizando, assim, seu particionamento na mesma ideia dos *shards*.



Saiba mais

O componente WebSphere ObjectGrid da IBM é um recurso ativado para armazenamento de dados na memória para objetos Java e projetado para escalabilidade, resiliência e alto desempenho. Ele pode ser usado simplesmente como um banco de dados de memória cache ou para distribuir dados através de uma rede. O ObjectGrid adota uma arquitetura de segurança para autenticar e autorizar clientes. Essa arquitetura é aberta e permite que o ObjectGrid se integre a recursos externos de segurança em servidores (IBM, [2020?]).

Estratégias para *sharding*

Existes diferentes estratégias para fazer o *sharding* dos dados. Algumas ferramentas, como o MongoDB (banco de dados NoSQL orientado a documentos) permitem que o usuário selecione a estratégia que melhor se adapta as suas necessidades. O MongoDB (KEEP; INGO, 2020, tradução nossa) possibilita o *sharding* por intervalo, por *hash* e por zona, descritos a seguir:

- *Sharding* por intervalo: usa a chave de *shard* para particionar os dados entre os *shards*. Neste sentido, dados com valores da chave semelhantes possivelmente estarão no mesmo *shard*. Bom para aplicações que utilizam muitas consultas sequenciais. A Figura 2 apresenta um exemplo de *sharding* por intervalo, onde a idade das pessoas foi utilizada para dividir os dados em três *shards*.
- *Sharding* por *hash*: distribui os dados de acordo com um *hash* MD5 do valor da chave do *shard*. Garante uma boa distribuição dos dados pelos *shards*. Bom para ingestão de *stream* de dados e dados de eventos.
- *Sharding* por zona: possibilita que o desenvolvedor especifique regras para orientar o armazenamento dos dados nos *shards*. Possibilita que uma empresa crie regras para separar dados de diferentes filiais em diferentes *shards*, por exemplo.

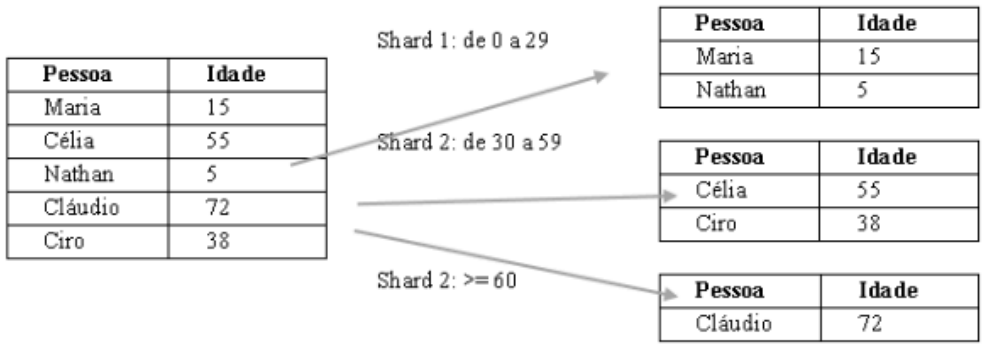


Figura 2. Exemplo de *sharding* por intervalo.

Assim como o MongoDB, outras ferramentas possibilitam o uso de diferentes estratégias para *sharding* dos dados, como o *sharding* baseado em diretório, baseado em colunas (útil para bancos de dados orientados a coluna como o Apache HBase), *sharding* dinâmico (quando o cliente determina onde será armazenado o *shard*) e algorítmico (quando se implementa uma função para alocação dos dados nos *shards*).

2 Utilizando *sharding* em *clusters*

O *sharding* é um método para distribuir dados em várias máquinas. Diferentes tipos de sistemas distribuídos usam *sharding* para oferecer suporte a implantações com conjuntos de dados muito grandes e operações de alto rendimento. Os sistemas com grandes conjuntos de dados ou aplicativos de alto rendimento podem desafiar a capacidade de um único servidor. Por exemplo, um volume muito grande de consultas em um mesmo intervalo de tempo pode esgotar a capacidade da CPU do servidor. Os tamanhos dos conjuntos de trabalho maiores que a RAM do sistema enfatizam a capacidade de entrada e saída das unidades de disco.

Uma das maneiras de controlar o consumo dos recursos de servidores é a utilização de *clusters* em servidores de dados. *Cluster* é um termo da língua inglesa que pode ser traduzido como “aglomerar” ou “aglomeração”. Em computação, é o termo que define um sistema que une vários computadores

em comum a fim de transformar o seu conjunto como se fosse uma única máquina com a soma de todas as capacidades reunidas (GOMES, 2015). A Figura 3 ilustra como funciona esse esquema.

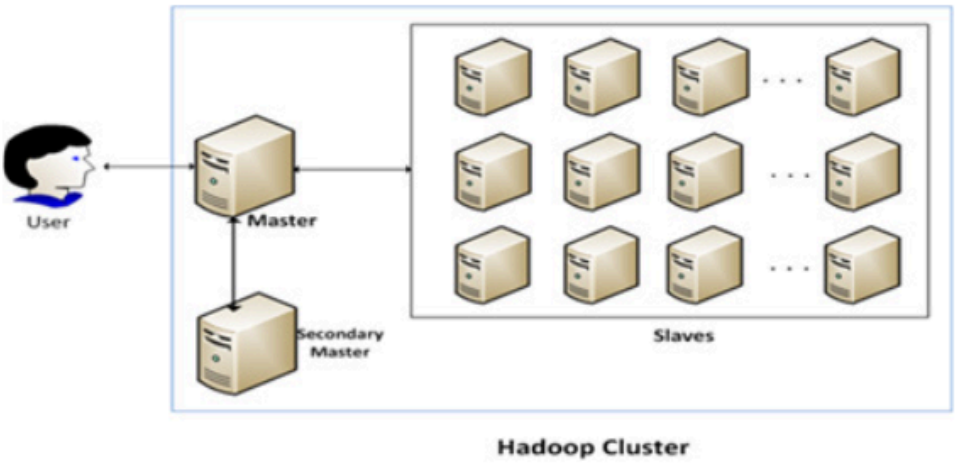


Figura 3. Esquema de *clusters*.

Conforme a imagem, existe um computador central, também chamado de *node master* ou nó mestre, e outros ligados a ele, os chamados *nodes* ou nós. Cada nó representa um computador acoplado a esse esquema que, ao se juntarem em nível de *software* dentro de uma rede, são vistos por computadores externos como sendo uma só máquina (*Slaves*). Cada membro de um *cluster* fragmentado deve poder se conectar a todos os outros membros no *cluster*. Isso inclui todos os *shards* e servidores de configuração. É preciso verificar se os sistemas de rede e segurança, incluindo todas as interfaces e *firewalls*, permitem essas conexões.

O uso da arquitetura em *cluster* é variado, pois ela é utilizada, hoje, tanto para atender à ideia de sistemas distribuídos, à fragmentação de dados (*sharding*) como também para a formação dos chamados supercomputadores, que são essenciais no processamento de algoritmos complexos com grandes volumes de dados e bastante utilizados em projetos de inteligência artificial. Um exemplo claro do uso de *clusters* ocorre na mineração de dados e, conforme Sharda, Delen e Turban (2019, p. 268), "a análise de agrupamento é um método essencial de mineração de dados para classificar itens, eventos ou conceitos em aglomerados comuns chamados de *clusters* ou agrupamentos".

O *load balancing* ou balanceamento de carga é uma prática em estruturas de *cluster* como forma de garantir que tarefas de processamento trabalhem de

maneira mais uniforme. Na prática, é a possibilidade de configurar que cada requisição que chegue ao servidor seja direcionada para um nó (ou computador) diferente, com objetivo de não sobrecarregar sempre a mesma máquina para todas as requisições. O balanceamento tem sua importância justamente na tentativa de garantir um equilíbrio no atendimento às requisições dos usuários. Sobre seu emprego, esse recurso é de uso variado, mas uma forma bastante comum de uso é em servidores *web* ou de *proxy* em uma rede local, pois a tarefa principal é garantir o acesso de seus usuários à grande rede, o que requer monitoramento constante.



Exemplo

Uma ferramenta que utiliza o *sharding* para distribuição dos dados em *cluster* é o ElasticSearch. Ele é comumente considerado um banco de dados não relacional (NoSQL), mas foi criado para gerenciar e funcionar como um motor de busca, como os que são utilizados pela Google, Yahoo, entre outros. A Elastic, empresa por trás do ElasticSearch, é especializada em ferramentas de controle de tráfego em servidores, como o Logstash e o Beats, que facilitam a coleta, agregação e enriquecimento de dados, armazenando-os no ElasticSearch. Com o Kibana, toda a *stack* da Elastic permite explorar, visualizar e compartilhar informações de maneira interativa sobre seus dados, além de gerenciar e monitorar a pilha. No ElasticSearch, acontecem todas as operações de indexação, fragmentação, pesquisa e análise.

Acesse a ferramenta no *link* a seguir (ELASTIC, [2020?]).

https://qrgo.page.link/whqHE

Geralmente, os *clusters* são implantados para melhorar o desempenho e a disponibilidade em relação a um único computador, além de serem muito mais econômicos do que computadores individuais com velocidade ou disponibilidade comparáveis. *Clusters* de computadores surgiram como resultado da convergência de várias tendências de computação, incluindo a disponibilidade de microprocessadores de baixo custo, redes de alta velocidade e *software* para computação distribuída de alto desempenho. Eles possuem uma ampla variedade de aplicabilidade e implantação, variando de *clusters* de pequenas empresas com vários nós a alguns dos supercomputadores mais rápidos do mundo.

Cluster local e remoto

Um *cluster* pode ser configurado em uma rede local ou também por acesso externo, ou seja, remoto. Enquanto um *cluster* local é, geralmente, conectado em redes do tipo LAN, *clusters* remotos são usados em recursos que funcionam se conectando por meio de *clusters* na camada de transporte. Os sistemas de computação em *cluster* podem estar entre os recursos mais valiosos de propriedade de uma organização. Como resultado, eles são alvos bastante visados por invasores e *hackers* e é essencial que eles estejam bem protegidos. Embora haja uma variedade de soluções de segurança para redes corporativas e máquinas individuais, o número de pesquisas focadas especificamente na proteção de sistemas de *cluster* tem crescido, mesmo que ainda não atenda aos requisitos que se apresentam ao lidar com *big data* e sua grande importância.

Com um *cluster* local, o nível de segurança pode ser melhor garantido devido à possibilidade de controle da aplicação como um todo, uma vez que ela pode estar com seu acesso limitado a quem estiver dentro da rede local. Porém, um componente importante na configuração de um *cluster* é permitir que o *software* de monitoramento de segurança da rede detecte uma classe inteira de ataques vindos de um usuário local autorizado invadindo de maneira externa. Detectar esse tipo de ataque é uma das muitas responsabilidades dos profissionais responsáveis pela criação e implantação de *clusters* em servidores de empresas e organizações de toda espécie.

Um módulo de *clusters* remotos permite estabelecer conexões unidirecionais para um *cluster* remoto. Essa funcionalidade é usada na replicação e na pesquisa de *cluster* cruzado. As conexões de *cluster* remoto funcionam configurando um *cluster* remoto e se conectando apenas a um número limitado de nós nele. Cada *cluster* é referenciado por um nome e uma lista de nós de semente. Quando um *cluster* remoto é registrado, seu estado de *cluster* é recuperado de um dos nós iniciais e até três nós de *gateway* são selecionados para serem conectados como parte das solicitações de *cluster* remoto.

Nesse contexto, existem os serviços em nuvem. Empresas como Google, Amazon e Microsoft apresentam soluções em nuvem para que empresas e projetos possam utilizar os recursos de seus potentes servidores como solução para clusterização de suas aplicações com relação a dados e arquivos em ambiente distribuído. A grande vantagem é que o custo diminui com a adesão a esses serviços, visto que despesas como energia e manutenção física dos servidores são totalmente terceirizadas, fazendo com que a equipe de TI de uma empresa foque apenas em criar as soluções para armazenamento e transações com os dados ali armazenados.



Fique atento

Existem diferentes tipos de clusters. *Clusters* podem ser utilizados em contextos diferentes, com estruturas de computadores das mais modestas até grandes servidores. Antes da criação dos *clusters*, *mainframes* — computadores de grande porte capazes de armazenar grande número de informações — eram adotados como uma grande máquina com redundância modular. Porém, o menor custo inicial dos *clusters* e o aumento da velocidade das redes favoreceram a adoção de *clusters* em empresas de vários portes. Ainda hoje, algumas empresas se utilizam de *mainframes*, mas esse número tem caído a cada ano.

Quando hospedamos uma solução na nuvem, ela pode ser configurada em um servidor tradicional ou em um *cluster* remoto. Enquanto os servidores tradicionais na nuvem podem ser máquinas virtuais localizadas em um único servidor físico, um cluster remoto é configurado sobre diferentes máquinas físicas. Neste sentido, a arquitetura de um *cluster* remoto pode ser semelhante à de um *cluster* local.

No Quadro 1, temos um exemplo comparativo entre servidores *cloud* padrão e servidores clusterizados, em uma empresa que fornece serviços de computação em nuvem (*Applied Innovations*). A empresa do exemplo ressalta que no serviço oferecido com servidores clusterizados, há a questão do *failover* automático, que trata-se da reconfiguração automática do sistema para colocar um novo nodo em serviço caso haja falha em algum dos nodos da rede.

Quadro 1. Exemplo comparativo entre servidores na nuvem.

	Servidores *cloud* padrão	Servidores *cloud* Clusterizados
# Servidores físicos	1	12-15
Tipo de armazenamento	RAID-10	SAN (Storage Area Network)
Failover Automatico	Não	Sim

Fonte: Adaptado de Cloud Servers... ([2020], tradução nossa).

Uma outra forma de configurar um *cluster* (seja ele local ou remoto), é a partir do uso de Kubernetes. Kubernetes é um sistema que facilita a orques-

tração de containers, como Dockers, possibilitando a criação de *clusters* virtualizados. As grandes empresas de nuvem, como AWS e Google Cloud possibilitam e facilitam a configuração de *clusters* remotos usando Kubernetes. Com uma ferramenta de orquestração, é transparente para o usuário em qual máquina ele está executando um processo, pois o Kubernetes gerencia os recursos de forma otimizada. Ou seja: pode haver mais de um nodo em uma máquina física, ou os nodos podem estar divididos pelo *cluster*, e o usuário não precisa se preocupar com isso. A Figura 4 a seguir mostra um exemplo do Google Kubernetes Engine (GKE), um *cluster* remoto baseado na nuvem da Google. Neste *cluster*, o Kubernetes se conecta a um painel de controle e a um conjunto de nós que são conectados aos serviços da Google Cloud, como o *Load Balancer* (para balanceamento de carga).

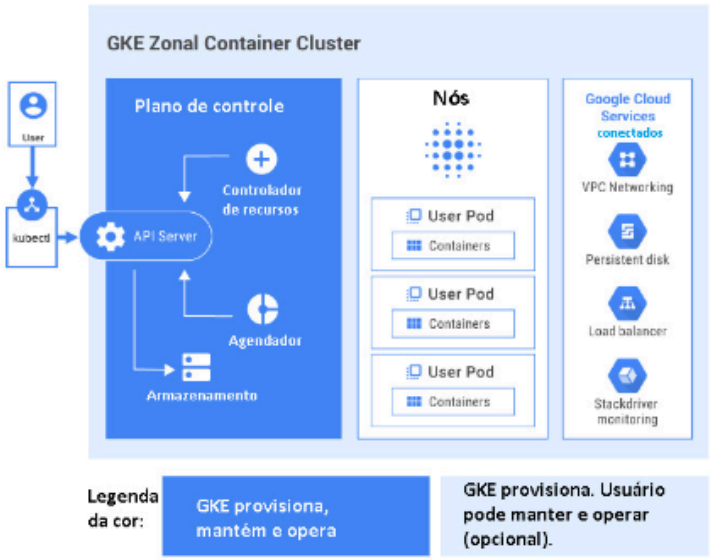


Figura 4. Google Kubernetes Engine (GKE).

Fonte: Arquitetura do cluster ([2020], documento ***on-line***).

A implementação de *shards* em *cluster* implica utilizar sistemas distribuídos que tenham a capacidade de empregar a técnica de *sharding*. Em um sistema de armazenamento de *big data*, cada *shard* contém um subconjunto dos dados fragmentados. Cada *shard* pode ser implantado como um conjunto de réplicas. O sistema distribuído atua como um roteador de consulta, fornecendo uma interface entre aplicativos clientes e o cluster *sharded*. Nessa operação, os servidores de configuração armazenam metadados e definições de configuração para o *cluster*. Há diferenças de um sistema de armazenamento de dados para

outro e, portanto, as configurações e o modo de colocar em prática o *sharding* também será feito de maneiras diferentes em cada um.

Existem requisitos a serem levados em consideração antes de realizar uma operação como essa. A complexidade da infraestrutura de *cluster* fragmentada exige planejamento, execução e manutenção cuidadosos. É necessária uma consideração cautelosa na escolha da *shard key* (chave de fragmento) para garantir o desempenho e a eficiência do *cluster*. Não se pode alterar a chave de fragmento após o fragmento, nem desfazer uma coleção de fragmentos.

3 Relação entre *sharding* e índices

Sistemas de armazenamento de dados possuem diversas formas de agrupar e armazenar seus dados. Isso também interfere no modo como os dados serão posteriormente acessados e pesquisados, o que gera uma cadeia de acontecimentos que são impactados, como o tempo de resposta e a integridade dos dados. Além do *sharding*, existem outras formas de organizar e trazer os dados em uma consulta ou transação com bancos de dados.

Tradicionalmente, sistemas de armazenamento de dados do tipo relacional utilizam índices como forma de organizar os registros de dados nas tabelas em operações de recuperação de dados. Ramakrishnan e Gehrke (2011) acrescentam que por meio de um índice é possível recuperar de forma eficiente todos os registros que satisfaçam alguma condição de pesquisa nos campos de chave do índice. A ideia central é que um índice sirva como um ponteiro direto para um registro específico, diminuindo o tempo de acesso a um dado. Como exemplo, é possível criar índices para consultar dados, como um somatório dos salários de funcionários de uma empresa. Para bancos de dados relacionais, índices podem ser primários ou secundários. Em índices primários, os registros devem incluir a chave primária, enquanto os demais registros sem a chave primária irão compor os índices secundários (RAMAKRISHNAN; GEHRKE, 2011).

Utilizando índices em bancos de dados

Segundo Bianchi (2007), outra forma de utilizar índices é para identificar dados em colunas, o que pode ajudar nas consultas que contenham a cláusula WHERE da linguagem SQL, que, ao utilizar operadores como NOT, AND e OR, acaba por alterar o desempenho das consultas. Ramakrish-

nan e Gehrke (2011) também ressaltam que existem diferentes tipos de índices, como:

- booleano;
- de *hashing*;
- *bitmap*;
- de várias tabelas;
- em árvore B (B-*tree*);
- funcional.

A Figura 5 ilustra a ideia do uso de índices em bancos de dados relacionais.

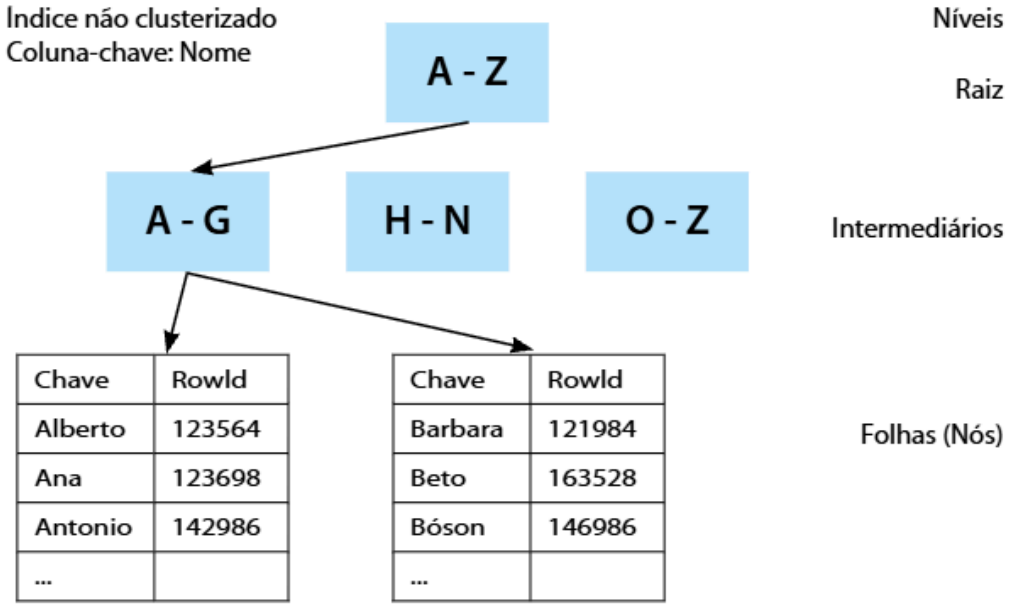


Figura 5. Utilizando índices em bancos de dados relacionais.

Fonte: Adaptada de Reis (2019).

Esse é um exemplo de estrutura de um índice não clusterizado do tipo B-*-tree* ou árvore balanceada. Nele, ao utilizar o algoritmo chamado de árvore balanceada, o desempenho de consultas aumenta, pois a busca por um nome é feita em um conjunto de dados menor, separado por categorias representadas pela primeira letra de um nome, filtrado pelo nome da pessoa, que é a coluna com valor como sendo a chave indexada. Como exemplo, ao consultar os dados de uma pessoa chamada Maria, ao usar essa coluna junto da cláusula WHERE em SQL, a consulta será feita no nó mestre, que, por sua vez, determina o

próximo nó a ser acessado, no caso, o nó intermediário H-N, onde estaria a letra M, inicial do nome de Maria.

Como é possível perceber, o uso de índices tem auxiliado muito na produtividade de equipes de desenvolvimento de sistemas no projeto de bancos de dados e em sua futura utilização. Afinal, é desde o projeto que se deve tentar prever o aumento do consumo dos dados e dos serviços oferecidos por servidores de dados. Pela falta de planejamento, muitos projetos do passado e alguns ainda no presente sofrem quando precisam aumentar a capacidade de usuários que os utilizam ao mesmo tempo. Em meio ao planejamento, a escolha entre um banco de dados que possa ser utilizado de maneira distribuída é uma possibilidade a ser considerada. A escolha entre o uso de índices ou a divisão do banco de dados em *clusters* desde o início, que auxilia no emprego da técnica de *sharding*, pode ser um fator decisivo no futuro dos dados de uma organização.

O Quadro 2 apresenta uma relação entre algumas das principais características do uso de índices e de *sharding* em sistemas gerenciadores de bancos de dados.

Quadro 2. Características de índices e *sharding*

Índices	*Sharding*
Melhora as consultas na maioria dos casos	Realiza o balanceamento de carga em servidores
Requer menos estrutura (tabela inteira em um mesmo servidor)	Utilizado em servidor clusterizado (várias máquinas interligadas trabalhando em conjunto)
Permite o acesso a dados ordenados sem precisar realizar a ordenação (dados já são armazenados de maneira ordenada)	Permite a replicação de dados com *shards* redundantes (alguns *shards* contendo o mesmo dado que outro como forma de garantir os dados sempre disponíveis)
Acesso aos dados feito por um dos três tipos de campos: chave primária, chave estrangeira e *Constraint Unique*	Acesso aos dados feito através de *shard keys* (chaves de partição)

Fonte: Adaptado de Oracle ([2020?]).

Portanto, índices e *sharding* possuem características bem específicas, mas ambos são úteis na melhoria do desempenho das consultas em um sistema de arquivos. Porém, ao mesmo tempo, a capacidade de escalabilidade do uso de *clusters* ou a simplicidade do armazenamento único em banco de dados não distribuído dos sistemas tradicionais são fatores que precisam ser colocados na balança pela equipe responsável em criar um projeto de implementação e manutenção de servidores de dados.



Fique atento

Utilizar índices em bancos de dados não tem apenas vantagens. Algumas desvantagens ou contrapartidas são notadas em certas situações. Por ser desenvolvida uma tabela virtual com todos os índices criados, aumenta o espaço de armazenamento em disco no servidor de banco de dados. Outra situação é que toda vez que um índice precisar ser alterado, a tabela com os índices também precisa sofrer alterações. Em alguns casos, o uso de índices pode até prejudicar o desempenho das consultas de bancos de dados, pois o uso dos índices pode fazer com que se demore mais tempo procurando a partição onde o dado está do que em uma consulta direta.

No que se refere ao desempenho, o uso de cada um desses recursos pode variar. A verdade é que o uso de índices ou de *sharding* depende da situação em que se encontra o projeto de uso dos dados. Quando se trata de *big data*, é inegável a necessidade de preparar o ambiente de servidor para lidar com uma imensidão de dados vindos a todo instante. Para o cenário atual de grandes conjuntos de dados, trabalhar com *clusters* é o mais indicado. Portanto, *sharding* pode ser um fator diferencial no acesso à informação vinda de *big data*.



Referências

ARQUITETURA do cluster. *Google Cloud*, [2020]. Disponível em: https://cloud.google.com/kubernetes-engine/docs/concepts/cluster-architecture. Acesso em: 29 out. 2020.

BIANCHI, W. Entendendo e usando índices: parte 1. *DevMedia*, Rio de Janeiro, 2007. Disponível em: https://www.devmedia.com.br/entendendo-e-usando-indices-parte-1/6567. Acesso em: 21 fev. 2020.

CASTRO, J. C. C. Entenda o que é escalabilidade. *Portal Educação*, [2020]. Disponível em: https://www.portaleducacao.com.br/conteudo/artigos/estetica/entenda-o-que-e-escalabilidade/59026. Acesso em: 10 set. 2020.

CLOUD Servers vs. Clustered Cloud Servers: Which Should You Choose? *AppliedInnovations*, [2020]. Disponível em:https://www.appliedi.net/blog/cloud-servers-vs-clustered-cloud-servers-which-should-you-choose/. Acesso em: 29 out. 2020.

ELASTIC. *Elasticsearch*. Versão 7.6. [*S. l.*]: Elastic, [2020?]. 1 programa de computador. Disponível em: https://www.elastic.co/pt/elasticsearch. Acesso em: 21 fev. 2020.

GOMES, P. C. T. O que é e como funciona um cluster? *OPServices*, Porto Alegre, 3 mar. 2015. Disponível em: https://www.opservices.com.br/o-que-e-um-cluster. Acesso em: 21 fev. 2020.

HIBERNATE SHARDS. Horizontal partitioning with hibernate: reference guide. *Docs*, [*s. l.*], [201–?]. Disponível em: https://docs.jboss.org/hibernate/stable/shards/reference/en/html_single/. Acesso em: 21 fev. 2020.

IBM. IBM websphere application server. *IBM*, [*s. l.*], [2020?]. Disponível em: https://www.ibm.com/cloud/websphere-application-server. Acesso em: 21 fev. 2020.

KEEP, M.; INGO, H. Performance Best Practices: Sharding. *MongoDB*, 18 fev. 2020. Disponível em: https://www.mongodb.com/blog/post/performance-best-practices-sharding. Acesso em: 26 set. 2020.

ORACLE. Oracle sharding. *Oracle*, [*s. l.*], [2020?]. Disponível em: https://www.oracle.com/database/technologies/high-availability/sharding.html. Acesso em: 21 fev. 2020.

RAMAKRISHNAN, R.; GEHRKE, J. *Sistemas de gerenciamento de banco de dados*. 3. ed. Porto Alegre: AMGH, 2011.

REIS, F. dos. O que são índices em bancos de dados: indexação em tabelas. *Bóson Treinamentos em Tecnologia*, [*s. l.*], 15 out. 2019. Disponível em: http://www.bosontreinamentos.com.br/bancos-de-dados/o-que-sao-indices-em-bancos-de-dados-indexacao-em-tabelas/. Acesso em: 21 fev. 2020.

SLICE. *OpenJPA for distributed object persistence*. [200–?]. 1 esquema. Disponível em: http://people.apache.org/~ppoddar/slice/site/index.html. Acesso em: 21 fev. 2020.

TANENBAUM, A. S.; BOS, H. *Sistemas operacionais modernos*. 4. ed. São Paulo: Pearson Education do Brasil, 2016.

VERAS, M. *Virtualização*: tecnologia central do datacenter – 2. ed. Rio de Janeiro: Brasport, 2016.



Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Conteúdo:

SOLUÇÕES EDUCACIONAIS INTEGRADAS

Dica do Professor: Sharding

Lidar com grandes volumes de dados não é tarefa fácil. Como forma de balancear a carga de um servidor de dados, alguns projetos de sistemas incluem, em sua infraestrutura, a utilização de *clusters* como forma de gerenciar essa carga de requisições. Com o *sharding*, é possível equilibrar a forma em que os dados serão alocados em servidores, utilizando os nós de um *cluster* como forma de alocação de dados de maneira particionada.

Na Dica do Professor, você saberá mais sobre o uso de *sharding* em servidores clusterizados em sistemas bancários.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Saiba mais: Sharding

Para ampliar o seu conhecimento a respeito desse assunto, veja abaixo as sugestões do professor:

Padrões e diretrizes arquiteturais para escalabilidade

Este artigo apresenta uma reflexão sobre a necessidade de se estabelecer arquiteturas com foco na escalabilidade, passando por aspectos como o alto desempenho de sistemas, aumento de sua capacidade, estrutura de servidores, entre outros aspectos.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

O que são Índices em Bancos de Dados – Indexação em Tabelas

Neste artigo, o autor apresenta o conceito de índices em bancos de dados relacionais, associando à indexação em tabelas no modelo relacional. Como conceito, o autor apresenta o embasamento teórico do tema, além de mostrar prós e contras dessa prática.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Infográfico: Frameworks que utilizam sharding como forma de distribuição

Apache Cassandra é um sistema de banco de dados muito popular que tem como características marcantes a descentralização, a tolerância a falhas e a alta escalabilidade. Uma das formas de garantir a escalabilidade é por meio do *sharding*.

Veja, no Infográfico a seguir, uma explicação resumida sobre a escalabilidade.

SHARDING NO APACHE CASSANDRA

O Apache Cassandra é um **banco de dados não relacional** cuja arquitetura é baseada no BigTable, no Amazon Web Service e no DynamoBD.

O particionamento horizontal de dados (*sharding*) é um método que aumenta a escalabilidade do sistema.



Veja, a seguir, mais informações sobre o Apache Cassandra.

HASH CONSISTENTE

O Cassandra utiliza um *hash* consistente para a replicação (3x) e o particionamento de dados. Os *hashs* são organizados em um anel de *token*.

MECANISMO DE BUSCA

Os dados são desnormalizados e duplicados, criando várias versões de uma mesma tabela, o que facilita o mecanismo de consulta.

ESCALABILIDADE

A escalabilidade se torna **quase infinita**, com a vantagem de que, se dobrar o número de nós, o sistema poderá lidar com quase o dobro de leituras e gravações.

Assim, o Cassandra se torna um sistema de banco de dados com alta disponibilidade de dados, rápida leitura, rápida gravação de dados e potencial de escalabilidade linear.

Conteúdo do Livro: Frameworks que utilizam sharding como forma de distribuição

Um bom *framework* de armazenamento de dados para *Big Data* precisa cumprir alguns requisitos, como tolerância a falhas, alta disponibilidade e escalabilidade. Em algum nível, a maioria dos bancos é escalonável, mas o diferencial é até onde essa capacidade vai.

Alguns deles permitem o acréscimo de máquinas para se tornar mais robusto, enquanto outros promovem também o particionamento de dados a fim de aumentar a sua capacidade. Este último tipo tem como vantagem uma escalabilidade quase infinita com menor custo, embora seja mais complexo que o primeiro.

No capítulo *Frameworks* que utilizam *sharding* como forma de distribuição, da obra *Framework de Big Data*, você vai conhecer a arquitetura do particionamento de dados em bancos em geral e também como isso acontece mais detalhadamente no Apache Cassandra e no MongoDB.

Boa leitura.

FRAMEWORK DE BIG DATA

Mariana Araújo Pereira



SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS

Frameworks que utilizam *sharding* como forma de distribuição

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Descrever a arquitetura dos *frameworks* que utilizam *sharding*.
- Definir como funciona o *sharding* no Apache Cassandra.
- Ilustrar como funciona o *sharding* no MongoDB.

Introdução

O *sharding* é uma forma de particionamento que possibilita o crescimento do banco de dados de forma horizontal e praticamente infinita. Assim, caso a demanda de armazenamento e processamento aumente de forma rápida, é possível manter as funcionalidades e a disponibilidade de dados presentes nesse banco consequentemente aumentando o número de máquinas. Atualmente, o *sharding* já é incorporado em alguns bancos de dados distribuídos.

Neste capítulo, você vai conhecer a estrutura básica do particionamento horizontal conhecido como *sharding* e vai aprender de que modo ele é implementado nos populares bancos de dados Apache Cassandra e MongoDB.

1 Arquitetura do *sharding*

Naturalmente, a tendência de um banco de dados é sempre crescer. Em alguns casos, esse crescimento pode chegar ao ponto em que a estrutura inicial das máquinas já não é capaz de atender à demanda, sendo necessário

realizar melhorias. Uma das opções é realizar um redimensionamento vertical, melhorando as máquinas em si de forma que possuam mais memória RAM e espaço em disco, por exemplo. O problema é que essa alternativa, além de trabalhosa, pode se tornar muito cara se for realizada localmente e, mesmo em casos de armazenamento em nuvem, essa solução não é infinita. Por conta dessas questões, é interessante recorrer a opções de bancos de dados que possibilitem um dimensionamento horizontal, no qual, ao invés de melhorar as máquinas existentes, mais máquinas são adicionadas e o conjunto de dados é particionado e redistribuído. Esse particionamento também é conhecido como ***sharding***.

O *sharding* possibilita o aumento da capacidade de armazenamento e processamento de um conjunto de dados, que já não são necessariamente armazenados em um único local ou servidor. Nesse método, os dados são divididos em pequenas partições (*shards*) e distribuídos em vários servidores, que inclusive podem ser sempre adicionados ao conjunto (*cluster*). O agrupamento dessas partições permite o armazenamento de grandes conjuntos de dados e o crescimento da capacidade de armazenamento e tráfego de acordo com a demanda exigida, pois, embora uma única máquina possa não apresentar uma alta capacidade geral ou velocidade, o *cluster* delas promove isso, de forma a possibilitar alta capacidade e alta velocidade ao sistema. O gargalo da utilização do *sharding* é a necessidade de aumentar a complexidade operacional do seu sistema: já não é mais possível acessar os dados em um único local e é necessário gerenciar várias máquinas ao mesmo tempo. Funções como pesquisa de texto e análises de dados, por exemplo, são beneficiadas por esse método.

Os *shards* de um mesmo banco de dados podem estar localizados na mesma máquina (*coresident*) ou em máquinas diferentes (remotas). O particionamento *coresident* tem como vantagem a redução do tamanho dos índices individuais e da quantidade de ações de E/S (entrada/saída) que se faz necessária durante a atualização de registros. Já o particionamento remoto possibilita o aumento da largura de banda do acesso aos dados, por meio do aumento de memória RAM para armazenamento, e disponibiliza mais interfaces de rede e canais de E/S do disco. A Figura 1 ilustra as diferenças entre um banco de dados particionado e um não particionado.

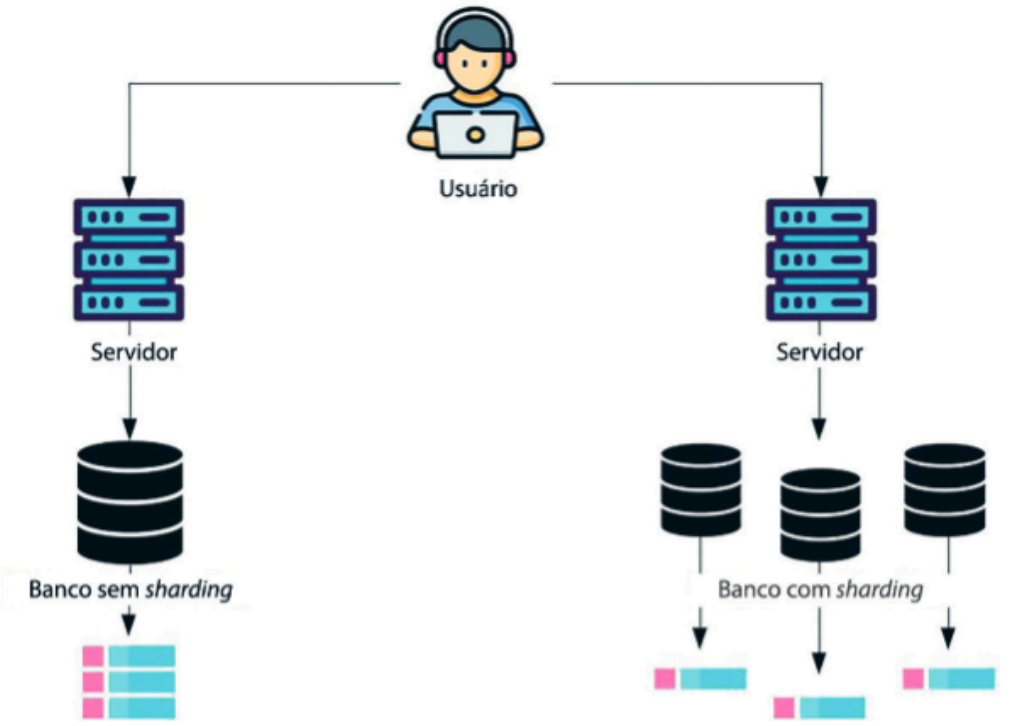


Figura 1. Exemplo de distribuição de dados em um banco não particionado e em um banco particionado (*sharding*).

Cada unidade de *shard* possui a mesma estrutura de banco de dados que o banco original e todos os *shards* juntos constituem o banco de dados lógico chamado de banco de dados particionados (SDB, *Sharded DataBase*). O SDB é a união de vários bancos de dados gerenciados coletivamente ou individualmente, de forma que, embora pareça único, o número de *shards* e a distribuição de dados são completamente transparentes para os aplicativos de banco de dados. Um *shard* é sempre gerenciado por um único servidor e, geralmente, há um esquema de replicação, em que o *shard* pode ser replicado duas vezes, por exemplo. Como vão acumulando dados ao longo do tempo, os *shards* podem ser configurados para passar por uma divisão automática quando chegam a um determinado limite de volume e redirecionar essas novas partições. O *sharding* automático reduz a carga da programação em comparação ao *sharding* manual, facilitando o gerenciamento do banco de dados quanto ao particionamento e à recuperação de dados do código do aplicativo.

Nesse processo de particionamento, a diferença dos conceitos de *sharding* e replicação pode não ficar tão evidente, mas na verdade eles são bem distintos. Enquanto o *sharding* é o particionamento de dados em fragmentos e os *shards*

armazenam subconjuntos desses dados, a replicação é a cópia exata de dados inteiros. A replicação de dados é extremamente importante, tendo em vista que sem ela há um ponto de falha no sistema, no qual, se um dos nós do *cluster* ficar inativo, parte dos dados não estará disponível. A criação de cópias dos dados (réplicas) supera esse desafio, tornando o sistema tolerante a falhas e mais confiável. Dessa forma, apesar de ter conceitos diferentes, apenas por meio da integração do particionamento e da replicação é possível obter um sistema de banco de dados eficaz. O Quadro 1 apresenta as diferenças entre replicação e particionamento.

Quadro 1. Diferenças entre replicação e particionamento

Replicação	Particionamento (*sharding*)
Armazena cópias dos dados em várias máquinas	Armazena partes de dados em várias máquinas
Possibilita o dimensionamento	Possibilita a redundância
Melhora o desempenho de leitura, mas é necessário realizar repetidamente a escrita/gravação de dados	Melhora o desempenho de leitura e de escrita/gravação de dados
Um conjunto de réplicas primário é o servidor único no conjunto capaz de receber gravações	Um *shard* primário é composto pelos conjuntos de réplicas que compõem um fragmento

Fonte: Adaptado de Bhamra (2017).

O conceito e a aplicação do *sharding* são antigos e, ao longo do tempo, diferentes tipos de estratégias e implementações desse método foram utilizados na construção de sistemas de grande escala. De forma geral, *sharding* é um tipo de arquitetura "nada compartilhada" (SN, *Shared Nothing*), em que não há centralização de dados e cada um dos fragmentos é independente. Os fragmentos são identificados a partir de chaves de fragmentação, que podem ser obtidas por estratégias distintas. Em relação às estratégias de chave de fragmentação no *sharding*, de acordo com Carpenter e Hewitt (2016), as três mais comuns são baseadas em: *hash*, intervalo e diretório, que serão detalhadas a seguir.

Sharding baseado em *hash*

O *sharding* baseado em *hash* (ou baseado em chave) utiliza valores constantes dos dados de cada fragmento para gerar um *hash* específico ou *token*, como é chamado em alguns bancos de dados. Esse novo *hash* é utilizado como uma chave de identificação que permite identificar quais dados serão armazenados em determinados *shards*. Por exemplo, na fragmentação de uma tabela cuja primeira coluna possui o nome completo de pessoas e esses nomes não se repetem, essa coluna poderia ser utilizada para criar uma chave de fragmentação e ser transformada em *hash*, para identificar cada porção fragmentada. Essa é uma arquitetura excelente para operações de dados direcionados, mas, além de o cálculo de *hash* poder adicionar uma carga adicional ao sistema, pode ser complicado adicionar ou remover servidores dinamicamente ao longo do tempo.

O valor do campo/coluna escolhido para ser a chave de fragmentação acaba por determinar o limite inferior inclusivo e o exclusivo de cada *shard*. Por exemplo, se um banco que possui identificadores que iniciam com números de 1 a 1000 (por exemplo: = 57ABD, 1GRS, 999ITR) fosse dividido em quatro *shards*, seria determinado que os *shards* armazenariam os ids em quatro grupos ordenados, podendo ser, respectivamente: S1: 1-250, S2: 251-500, S3:501-750 e S4: 751-1000. Sendo assim, além de determinar os limites dos *shards*, as chaves acabam também determinando o local em que cada fragmento ficará alocado. A cada adição, é verificado o lugar no qual a chave daquele fragmento melhor se encaixa para, então, alocá-lo. No caso de um *shard* ficar muito desproporcional aos outros, ocorre um redimensionamento dos dados. Essa chave também pode ser utilizada para as consultas, o que possibilita uma forma rápida de chegar a um fragmento específico.



Fique atento

Hash é uma sequência de bits que permite a visualização de dados por meio de letras e números em uma variável de comprimento fixo. Em outras palavras, é a transformação de uma grande quantidade de dados em uma pequena quantidade de informações. Este capítulo inteiro, por exemplo, pode ser representado pelo *hash*: ffb2ef732291fc82135c9ba765163ded.

Sharding baseado em intervalos

O *sharding* baseado em intervalos particiona os dados com base em "intervalos de chave", que correspondem aos intervalos de valores dos dados, de forma que o *shard* preserva o mesmo esquema do banco original e é possível identificar facilmente o intervalo dos dados correspondente a cada *shard*. Por exemplo: em um banco de dados de uma loja de produtos em um *shopping*, dentre as informações, há o preço de cada item. Em um *sharding* baseado em intervalos, esse banco poderia ser dividido de acordo com faixas de preço, de forma que cada *shard* guardaria todos os produtos correspondentes àquela faixa.

A estratégia de *sharding* baseado em intervalos torna a implementação relativamente fácil, mas é preciso estar atento ao desequilíbrio de cargas, à acessibilidade e velocidade de cada *shard*: a partir da definição de intervalos, é possível que o número de objetos (ou itens, no exemplo) seja diferente em cada parte fragmentada, o que pode interferir no rebalanceamento de cargas e no acesso aos dados. Por outro lado, essa é uma boa estratégia no caso do planejamento baseado em intervalo, como no exemplo dos preços, pois a consulta, basicamente, será realizada em um único *shard*, que é o que possui o preço do produto embutido em seu intervalo, em vez de ter que consultar todos os *shards*. Geralmente, essa é a estratégia de *sharding* padrão dos aplicativos.

Sharding baseado em diretório

O *sharding* baseado em diretório necessita de um serviço de pesquisa estática, que permita rastrear e identificar qual *shard* mantém quais dados, como um mapa. Nesse tipo de particionamento, o aplicativo cliente, primeiro, realiza uma consulta no serviço de pesquisa a fim de identificar o *shard* correto e, em seguida, esse *shard* é retornado. A principal vantagem desse tipo de particionamento é a flexibilidade, pois, ao invés de estar limitado ao uso de chaves ou intervalos fixos, é possível utilizar qualquer sistema ou algoritmo que se deseja atribuir dados a *shards*. Dessa forma, o mapeamento entre um *shard* virtual e uma partição física pode mudar sem a necessidade de modificar o código do aplicativo e é relativamente fácil adicionar *shards* dinâmicos. O principal problema do *sharding* baseado em diretórios é que, se o sistema de busca for corrompido, a capacidade de gravação e acesso de dados é afetada.

Além dessas três estratégias, a depender do banco de dados escolhido, também é possível fragmentar os dados de outras maneiras. Eles podem ser fragmentados de acordo com a carga de trabalho, localização, prioridade e

privacidade de acesso dos usuários ao sistema, por exemplo. Embora pareça ter apenas vantagens, o particionamento horizontal deve ser complementar a outras formas de particionamento, como o vertical e o funcional. Antes de escolher esse tipo de particionamento, independentemente da estratégia, de forma geral, é necessário estar atento a alguns pontos, como a escolha da chave de fragmentação, o equilíbrio e o redimensionamento, e a eficiência de consultas de dados.

Os *shards* devem ter o volume de dados o mais semelhante possível entre si. Como a soma ou a remoção de dados podem ser comuns no banco, é necessário que haja um rebalanceamento periódico a fim de garantir uma distribuição uniforme e reduzir a chance de geração de problemas. Como esse processo pode ser oneroso, o ideal é planejar o crescimento do banco, de forma que em cada *shard* haja o espaço necessário para as alterações esperadas. É necessário também ter uma equipe capaz de desenvolver estratégias para reequilibrar rapidamente os *shards*, sempre que necessário.

Os dados que serão utilizados como chave de fragmentação devem ser escolhidos cuidadosamente, pois precisam ser informações estáveis e pouco voláteis. Isso porque, se a chave de fragmentação for alterada, talvez seja necessário mover o fragmento correspondente, o que aumenta as operações de atualização e a movimentação de dados. Por isso, a informação escolhida como chave de fragmentação deve ser, preferencialmente, invariável e exclusiva.

A eficácia dos sistemas de consulta varia de acordo com a estratégia de chave de fragmentação que será utilizada. Aquelas nas quais é preciso acessar apenas um *shard* são mais eficientes do que as que recuperam dados de vários *shards*. É por isso que se deve conhecer os dados e possíveis buscas, a fim de evitar a implementação de um sistema que acabe resultando em aplicativos que precisam executar grandes números de consultas para chegar ao resultado esperado. Além disso, é possível incluir a chave de fragmentação de um objeto como parte do esquema de um outro objeto, caso eles sejam referenciados entre si. Por exemplo, se houver diferentes resultados de exames de sangue de um paciente ao longo dos anos em um banco dividido por anos, é possível fazer referências interligando todos os resultados desse mesmo paciente. Assim, é possível melhorar o desempenho das consultas em bancos que possuem dados referenciados.

O ato de utilizar o processo de escalonamento horizontal, possibilitando distribuir uma grande quantidade de dados em diferentes *shards*, permite obter ganhos de desempenho horizontal, com alta disponibilidade de dados, tolerância a falhas, segurança, paralelização dos processos de *backup*, restau-

ração e sincronização inicial. Alguns bancos de dados famosos utilizam esse tipo de dimensionamento, como Cassandra (APACHE CASSANDRA, c2016), MongoDB (c2020) e Hbase (APACHE HBASE, c2020), além do sistema de arquivos HDFS do Apache Hadoop ([2019?]).

2 *Sharding* no Apache Cassandra

Apache Cassandra é um sistema de banco de dados distribuído projetado para lidar com grandes cargas de trabalho (*big data*). Ele foi projetado com base no DynamoDB (AMAZON WEB SERVICE, c2020), da Amazon Web Services (AMAZON WEB SERVICE, [2020?]) e do BigTable (GOOGLE CLOUD, [2020?]), de forma que a construção fi nal promove alta escalabilidade e tolerância a falhas. Cassandra possui uma arquitetura SN, já que não tem controlador central nem uma relação de mestre/escravo entre os nós, de forma que todos são iguais (CARPENTER; HEWITT, 2016).

No Cassandra, cada linha de uma tabela é referenciada por uma chave primária, também conhecida como chave de linha. Há um número de colunas em uma linha que pode variar em linhas diferentes. Assim, enquanto a linha 1 possui cinco colunas, a linha 2 pode possuir três. A linguagem de consulta do Cassandra (CQL, *Cassandra Query Language*) é baseada na linguagem SQL (*Structured Query Language*), seguindo o formato de tabelas, linhas e colunas. Para cada dado é criada uma chave primária, que é constituída de "chave de partição" e, opcionalmente, de colunas do *cluster*. A chave primária difere da chave de partição gerada pelos particionadores, de forma que essa última representa uma partição de dados exclusiva.

De forma geral, cada tabela do Cassandra possui uma chave de partição, que pode ser autônoma ou composta e determina a localidade dos dados por meio da indexação. Essa chave deve criar partições com tamanhos possivelmente ideais, a fim de minimizar a movimentação de dados para redimensioná-los. O tamanho dessa partição deve ser igual ou inferior a 10 MB, podendo ser até de 100 MB.

Nesse sistema, cada servidor é um *shard* com seus dados replicados por meio de outros servidores e, geralmente, a estrutura do *sharding* utilizada é chave/*hash*, que, nesse caso, é chamado de *token*. A partir dos *tokens*, os objetos são atribuídos a cada *shard* em partes iguais e distribuídos uniformemente no *cluster* principal. O Cassandra também replica os *shards* para outros nós no *cluster*, de acordo com as configurações de replicação predefinidas.

O balanceamento de cargas é simplificado a partir dos intervalos de *hash* que definem cada *shard*, deixando-os, em média, com um número igual de linhas.

Nesse sistema, existem três mecanismos para realizar o *sharding*, sendo que todos resultam na estrutura de *token*: `Murmur3Partitioner` (padrão), `RandomPartitioner` e `ByteOrderedPartitioner`, cuja diferença está relacionada à forma de gerar o código chave/*hash* de cada dado. `Murmur3Partitioner` possibilita a distribuição uniforme dos dados pelo *cluster* com base nos valores de *hash* `MurmurHash` e pode melhorar o desempenho de três até cinco vezes em relação aos outros. Já `RandomPartitioner` realiza a mesma função com base nos valores de *hash* MD5, mas, como ele é um *hash* criptográfico, a geração das chaves demora mais tempo. `ByteOrderedPartitioner`, por sua vez, mantém a distribuição ordenada por bytes-chave de forma léxica, mas não é muito utilizado. Esses particionadores não são compatíveis entre si, de forma que os dados particionados por meio de um particionador dificilmente serão convertidos em outro tipo de forma correta.

Hash consistente

No Cassandra, o processo de *hash* é um pouco diferente que o usual. Nesse sistema, há *hashing* consistente, que se distingue do *hash* comum porque, ao invés de alocar as chaves em "baldes" (*buckets*), o Cassandra armazena as chaves em uma estrutura anel contínuo. Isso permite que, ao invés de ter que remapear todos os *hashes* a cada acréscimo de objeto, o *hash* simplesmente seja movido para a próxima posição. O *hash* consistente é um processo de *hash* mais sofi sticado, em que um grande número de intervalos de *tokens* é distribuído igualmente entre os servidores. Cada linha armazena um pedaço de registros, com base em um *hash* de valores-chave que se encaixam no intervalo de um determinado servidor. Quando servidores são adicionados, fragmentos de registros podem ser migrados lentamente, reduzindo a quantidade de movimentação de dados no sistema. O *hash* consistente também minimiza os movimentos das *keys* durante os procedimentos de adição ou decréscimo de nós no *cluster*.

Na inicialização do *cluster* no Cassandra, cada nó recebe um intervalo de *token* que determina sua posição e a quantidade dos dados armazenados nele. Cada nó recebe um intervalo proporcional dos intervalos de *token* para garantir que eles sejam distribuídos uniformemente pelo anel (NAZEER *et al.*, 2017). De acordo com a documentação do Cassandra (APACHE CASSANDRA, [201-?]), é possível haver até 255 *tokens* em um nó. Caso haja adição de um novo fragmento, ele será adicionado ao nó cujo número do seu *token* está

contido no intervalo que determina o limite entre os nós. Por exemplo, no caso do acréscimo de um fragmento cujo *token* é 127, na estrutura da Figura 2, ele será armazenado no nó 4.

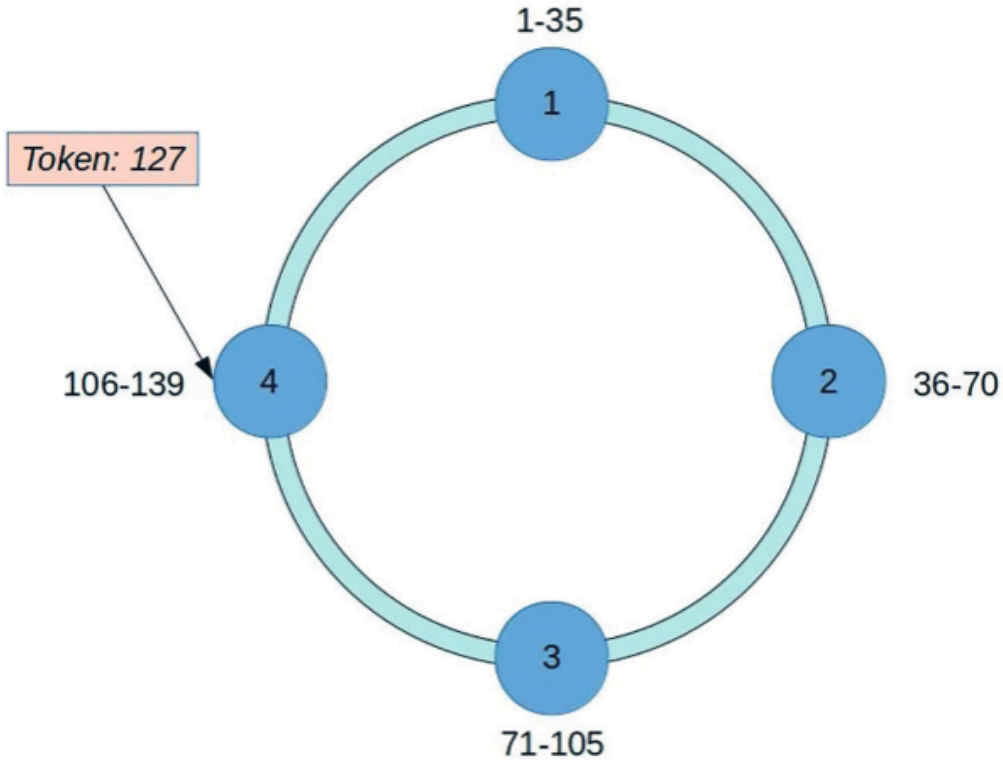


Figura 2. *Hash* consistente em estrutura de anel.

Quando uma solicitação de leitura ou gravação é recebida por qualquer nó no *cluster*, os seguintes passos são executados:

1. identificação dos nós que possuem os dados cujas chaves correspondem à busca;
2. encaminhamento das solicitações para os nós, seguido da espera pela resposta;
3. caso não haja retorno em um determinado valor de tempo limite pré-configurado, retorno de "falhar na solicitação" para o aplicativo cliente;
4. caso haja retorno, determinação da resposta mais recente de acordo com os registros de data e hora;
5. agendamento do reparo dos dados nas réplicas, caso haja modificações.

A integração da replicação e do particionamento promovem a alta disponibilidade de dados. A replicação do Cassandra acontece dentro da estrutura de anel. Nela, cada fragmento de dado é replicado em N *hosts*, em que N é o fator de replicação configurado "por instância" e cada chave é atribuída a um nó coordenador, que é responsável pela replicação dos fragmentos de dados que se enquadram dentro do seu alcance. O Cassandra elege um líder entre os nós do anel, utilizando uma ferramenta do sistema chamada Zookeeper (APACHE ZOOKEEPER, [2019?]). Assim, cada nó que ingressa no *cluster* entra em contato com o nó líder que, de certa forma, decide para quais outros nós ele vai ser replicado. Os seus metadados e os dos outros nós são armazenados em cache localmente (um para cada nó), de forma que são tolerantes a falhas (ainda utilizando o Zookeeper) (LAKSHMAN; MALIK, 2010).

Os protocolos utilizados para a realização da replicação e do particionamento são dependentes da identificação da atividade de cada *shard* no *cluster*. No Cassandra, as informações sobre um *shard* que sinalizam se ele está ativo (vivo) ou inativo (morto) são compartilhadas por um mecanismo de detecção de falhas baseado em "fofocas" (*gossip protocol*, em inglês, que literalmente pode ser traduzido para "protocolo de fofocas"). A partir desse protocolo, os nós recebem informações de estado sobre outros nós interligados (com o máximo de três), facilitando a detecção de falhas no sistema. Essa detecção também é usada para evitar tentativas de estabelecer uma comunicação com nós inacessíveis durante várias operações. O protocolo baseado em fofocas apresenta uma excelente precisão e ótima velocidade, além de se ajustar bem às condições de sobrecarga da rede e do servidor.

Além de indicar o estado de atividade de cada nó, o protocolo de fofocas também fornece informações a respeito da posição do nó, dos metadados e da esquematização. Após o posicionamento da partição em um dos *shards*, as informações associadas ao *token* e sua posição dentro do anel são fofocadas para todo o *cluster*, possibilitando que todos os nós possam ter as mesmas informações e desempenhar as mesmas funções, inclusive executar o algoritmo de inicialização (*bootstrap*). A fofoca acontece a cada segundo, em que cada nó/*shard* do *cluster* seleciona outro nó de forma aleatória para trocar informações sobre o estado de atividade.

A arquitetura simplificada (uma única máquina) do Cassandra para o particionamento por *sharding* necessita de, basicamente, quatro itens: módulo de particionamento, associação de *cluster*, módulo de detecção de falhas e módulo do mecanismo de armazenamento. Cada um desses módulos possui uma programação própria que se torna uma fase ou um estágio do processo em geral.

O procedimento de *sharding*, no sistema de armazenamento de bancos de dados do Cassandra, contribui para que o sistema apresente alta escalabilidade, alto desempenho e ampla aplicabilidade. Além disso, Cassandra pode suportar uma taxa de transferência de atualização de dados muito alta, enquanto fornece baixa latência, o que é uma vantagem que deve ser levada em conta durante a tomada de decisão sobre qual sistema utilizar.



Saiba mais

Leia mais sobre o Apache Cassandra, seus modelos de particionamento e replicação, no artigo "Apache Cassandra", de Kerlla de Souza Luz e Thiago Vinícius de Melo Almeida. O artigo está disponível no *site* do Departamento de Ciências da Computação, da Universidade de Brasília (UnB).

3 *Sharding* no MongoDB

O MongoDB, segundo os próprios desenvolvedores (MONGODB, c2008, documento *on-line*), é "um banco de dados distribuído, baseado em documentos e de propósito geral, desenvolvido para desenvolvedores de aplicativos modernos e para a era da nuvem". Esse banco suporta o particionamento automático de dados, o que simplifi ca a arquitetura do aplicativo e o seu manejo, utilizando a estrutura de chave/*hash*. Operacionalmente, esse banco de dados automatiza o balanceamento de dados entre os *shards* e facilita a adição e a remoção de máquinas que vão alterar a capacidade de armazenamento. São necessários três componentes principais para constituir o MongoDB por completo: *shards*, roteadores de consulta e servidores de confi guração. Os *shards* são conjuntos de réplicas de servidores que armazenam os subconjuntos de dados; os roteadores de consulta são a interface entre o aplicativo cliente e os *shards* propriamente ditos. Para consultar o *shard* correto, a consulta utiliza informações fornecidas pelos servidores de confi guração, que, por sua vez, armazenam apenas metadados dos *shards* que possuem a localização de cada dado e outras confi gurações gerais (BRADSHAW; BRAZIL; CHODOROW, 2019). A Figura 3 ilustra o que é necessário para um banco de dados particionado.

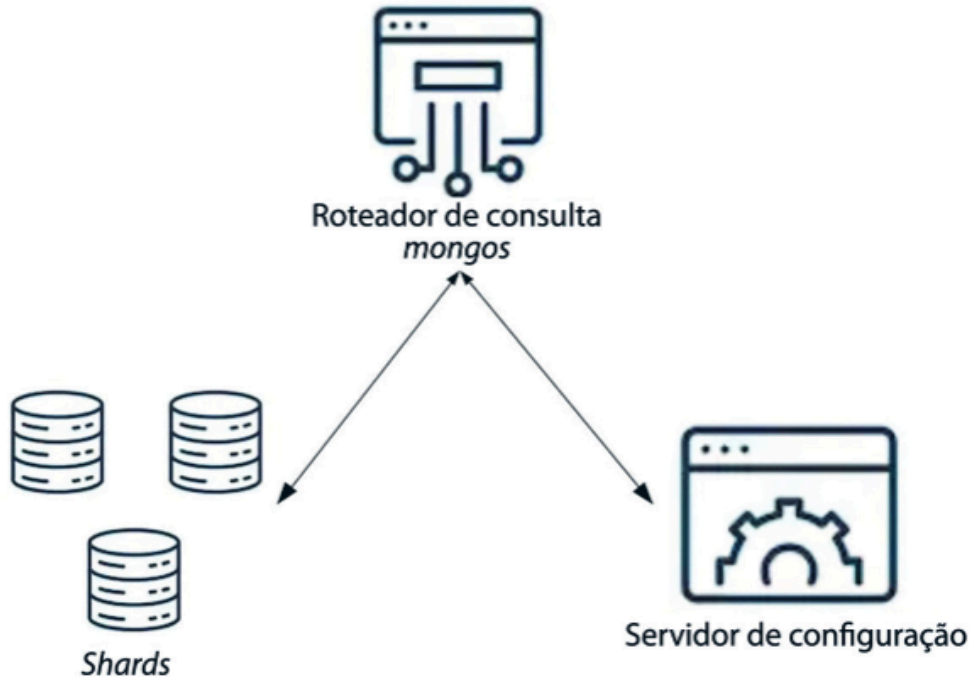


Figura 3. Diagrama do mínimo necessário para um banco de dados particionado (*sharded*).
Fonte: Adaptada de NikWB/Shutterstock.com.

Para cada conjunto de dados em um *cluster* fragmentado, há um *shard* primário, que contém toda a informação não fragmentada desses dados. Após a fragmentação, são criados *shards* secundários que possuem um conjunto de réplicas de si. Cada um dos *clusters* que contém o *shard* primário e o conjunto de réplicas deve possuir o próprio servidor de configuração, já que as operações realizadas nesses servidores podem ter um impacto significativo no desempenho e na disponibilidade do *shard*. Se, por acaso, o *cluster* de um determinado servidor de configuração perder o primário, os metadados do *cluster* se tornarão somente leitura. Dessa forma, será possível ler e gravar dados dos *shards*, mas as operações de migração ou divisão de fragmentos não ocorrerão até que um novo *shard* seja eleito para essa posição.

Em *clusters* que passaram pelo processo de *sharding*, é preciso configurar os roteadores de consulta, que são chamados de *mongos*. Os *mongos* utilizam as chaves primárias para executar a consulta solicitada pelo usuário. Em um banco de dados de uma universidade, por exemplo, as chaves podem ser o CPF dos alunos e as consultas podem ser realizadas a partir desse identificador em

shards que foram divididos a cada 1000 registros na ordem crescente. Para saber em qual *shard* está o arquivo que corresponde à consulta, os *mongos* utilizam os metadados de cada partição, a partir dos quais é possível, inicialmente, reduzir a busca, se o número buscado iniciar com 3, por exemplo. Esses metadados, que estão nos servidores de configuração, são mutáveis e talvez a razão mais comum dessas modificações seja quando é necessário fazer uma redistribuição de dados entre os *shards* no caso de um acréscimo ou decréscimo de informações no banco. Por isso, a cada consulta, os *mongos* acessam os metadados para identificar a localização do arquivo, mesmo que essa consulta já tenha sido realizada anteriormente.

Uma outra importante função dos *mongos* é a mesclagem de dados antes de retornar a consulta para o aplicativo cliente. Se no banco de dados exemplificado anteriormente, que foi dividido pelo número de CPF, for solicitada uma busca pelo valor de mensalidade X, essa consulta será realizada em todos os *shards* do *cluster*. Cada um dos *shards* enviará seus resultados aos *mongos*, nos quais serão reunidos e poderão ser classificados por outros critérios, se assim a consulta exigir. Após a mesclagem de todos os resultados de todos os *shards* ser finalizada, os *mongos* retornarão os resultados ao aplicativo cliente, como se fosse um único banco.

O MongoDB suporta as estratégias de *sharding* de chave/*hash* e de intervalo, que devem ser cuidadosamente escolhidas antes da fragmentação, já que a seleção de qual campo será utilizado para gerar chave de fragmentação é imutável. A escolha dessa chave pode afetar o desempenho, a eficiência e a escalabilidade de um *cluster* fragmentado, mesmo que ele possua o melhor *hardware* e a melhor infraestrutura possíveis. A escolha do que será utilizado como chave de fragmento e o seu índice de suporte também podem afetar a estratégia que poderá ser utilizada.

No caso da estratégia baseada em *hash*, o sistema calcula automaticamente cada um dos *hashes* dos fragmentos, o que gera um custo de sobrecarga. Nesse tipo de estratégia, a probabilidade de os dados buscados estarem contidos em um único fragmento é menor, resultando em mais operações de busca em todo o *cluster*, pois, possivelmente, ela deverá ser realizada em mais de um *shard*. Já nas situações em que a estratégia adotada é a de intervalo, a sobrecarga é menor, tendo em vista que essa é a estratégia padrão e há uma maior probabilidade de se encontrar todos os dados procurados em um mesmo *shard*, caso a busca esteja limitada a um determinado intervalo de valores.

O MongoDB possui alta tolerância e disponibilidade, de forma que um *cluster* fragmentado pode continuar executando as operações parciais de leitura e gravação mesmo quando um ou mais *shards* estão em estado inativo, embora os fragmentos armazenados nesses *shards* não possam ser acessados caso não haja réplicas. Caso o *cluster* possua um conjunto de réplicas do servidor de configuração (CSRS, *Config Server Replica Set*), é possível continuar o processamento de leituras e gravações enquanto a maioria desse conjunto de réplicas estiver disponível. Por isso, é muito importante manter a implementação do conjunto de réplicas, o que garante maior redundância e disponibilidade dos dados (BOAGLIO, 2015).

O *sharding* é um processo importante para dimensionar bancos de dados de forma horizontal, possibilitando o aumento da capacidade de processamento e armazenamento sempre que necessário. A principal diferença entre a arquitetura de *sharding* do MongoDB e do Cassandra é a forma de replicação dos dados. No Apache Cassandra, cada *shard* é um único servidor e um mesmo objeto é armazenado em vários *shards*. Já no MongoDB, cada *shard* é um conjunto de réplicas de vários servidores. Desse modo, cada objeto está contido apenas em um *shard*, embora replicado várias vezes. No caso de falha do servidor do Cassandra, outro *shard* conterá os dados, enquanto no MongoDB, dentro do próprio *shard*, outra unidade do conjunto de réplicas irá substituí-lo. Essas falhas associadas a esse processo resultarão em uma maior (Cassandra) ou menor (MongoDB) movimentação dos dados entre os nós, o que impacta em um desempenho menos ou mais estável desses sistemas de bancos de dados (BHAMRA, 2017).

O foco principal do processo de particionamento é melhorar o desempenho e a escalabilidade de um sistema e, como adicional, ainda possibilitar a melhora da disponibilidade, a depender de como esses dados serão divididos e replicados. Caso haja falha em uma partição, isso não significa que necessariamente um aplicativo cliente seja impedido de acessar os dados, que podem estar replicados em outras partições. Por isso, ao estudar a utilização do *sharding* como estratégia para o particionamento de bancos de dados, é preciso atenção quanto às formas de replicação que o banco permite, pois é com o bom manejo entre esses dois conceitos que o sistema de banco de dados escolhido pode ser aproveitado da melhor forma.



Referências

AMAZON WEB SERVICE. *Amazon DynamoDB*. [*S. l.*]: Amazon Web Service, c2020. Disponível em: https://aws.amazon.com/pt/dynamodb/. Acesso em: 10 abr. 2020.

AMAZON WEB SERVICE. *AWS*. [*S. l.*]: Amazon Web Service, [2020?]. Disponível em: https://aws.amazon.com/pt/?nc2=h_lg. Acesso em: 10 abr. 2020.

APACHE CASSANDRA. Architecture. *Apache Cassandra*. [*S. l.*], [201-?]. Disponível em: http://cassandra.apache.org/doc/latest/architecture/index.html. Acesso em: 9 abr. 2020.

APACHE CASSANDRA. *Cassandra*. Versão 3.11.6. [*S. l.*]: The Apache Software Foundation, c2016. Disponível em: http://cassandra.apache.org/. Acesso em: 10 abr. 2020.

APACHE HADOOP. *Apache Hadoop*. Versão 3.2.1. [*S. l.*]: The Apache Software Foundation, [2019?]. Disponível em: https://hadoop.apache.org/. Acesso em: 10 abr. 2020.

APACHE HBASE. *Apache HBase*. Versão 2.0. [*S. l.*]: The Apache Software Foundation, c2020. Disponível em: https://hbase.apache.org/. Acesso em: 10 abr. 2020.

APACHE ZOOKEEPER. *Apache Zookeeper*. Versão 2.0. [*S. l.*]: The Apache Software Foundation, [2019?]. Disponível em: https://zookeeper.apache.org/. Acesso em: 10 abr. 2020.

BHAMRA, K. *A comparative analysis of MongoDB and Cassandra*. 2017. Dissertação (Mestrado em Ciências) - University of Bergen, Bergen, 2017. Disponível em: http://bora.uib.no/bitstream/handle/1956/17228/kb-thesis.pdf?sequence=1&isAllowed=y. Acesso em: 9 abr. 2020.

BOAGLIO, F. *MongoDB*: construa novas aplicações com novas tecnologias. São Paulo: Casa do Código, 2015.

BRADSHAW, S.; BRAZIL, E.; CHODOROW, K. *MongoDB*: the definitive guide powerful and scalable data storage. 3rd ed. Sebastopol: O'Reilly Media, 2019.

CARPENTER, J.; HEWITT, E. *Cassandra*: the definitive guide distributed data at web scale. 2nd ed. Sebastopol: O'Reilly Media, 2016.

GOOGLE CLOUD. *Cloud Bigtable*. [*S. l.*]: Google, [2020?]. Disponível em: https://cloud.google.com/bigtable. Acesso em: 10 abr. 2020.

LAKSHMAN, A.; MALIK, P. Cassandra: a decentralized structured storage system. *ACM SIGOPS Operating Systems Review*, [*s. l.*], v. 44, n. 2, p. 35-40, 2010. DOI: https://doi.org/10.1145/1773912.1773922. Disponível em: http://www.cs.cornell.edu/Projects/ladis2009/papers/Lakshman-ladis2009.PDF. Acesso em: 9 abr. 2020.

MONGODB. *MongoDB*. [*S. l.*]: MongoDB, c2020. Disponível em: https://www.mongodb.com/. Acesso em: 10 abr. 2020.

MONGODB. Sharding reference. *MongoDB*, [*s. l.*], c2008. Disponível em: https://docs.mongodb.com/manual/reference/sharding/. Acesso em: 10 abr. 2020.

NAZEER, H. *et al.* Real-time text analytics pipeline using open-source big data tools. *Arxiv*, [*s. l.*], n. 1712.04344, 2017. Disponível em: https://arxiv.org/pdf/1712.04344.pdf. Acesso em: 10 abr. 2020.

Leitura recomendada

LUZ, K. S.; ALMEIDA, T. V. M. *Apache Cassandra*. 2016. Disponível em: https://cic.unb.br/~alchieri/disciplinas/posgraduacao/sd/artigoG2.pdf. Acesso em: 13 abr. 2020.



Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.

Conteúdo:

SOLUÇÕES EDUCACIONAIS INTEGRADAS

Dica do Professor: Frameworks que utilizam sharding como forma de distribuição

No mundo digital globalizado, alguns nichos de negócios requerem bancos de dados compartilhados e colaborativos a fim de registrar transações em todo o planeta. Para suprir essa demanda, surgiu a tecnologia de *blockchain*, que é muito utilizada principalmente no mercado das criptomoedas.

Confira, na Dica do Professor, como o *sharding* pode estar envolvido e qual a sua importância para a manutenção de um sistema desse tipo.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Atividade 3 (A3)

1) O particionamento horizontal de um banco de dados (ou *sharding*) é realizado utilizando estruturas que permitem a identificação de cada partição. Quanto a estas, as três mais comuns são:

A) chave, *hash* e *hash* consistente.

B) *hash*, zona e intervalo.

C) *hash*, intervalo e diretório.

D) chave, intervalo e faixas.

E) intervalo, zona e localização.

2) O particionamento de dados está intimamente ligado à replicação. Quantos a esses dois processos, é verdade que:

A) apenas o conceito está associado, mas na prática não há ligação entre eles.

B) apesar de terem nomes distintos, significam a mesma coisa.

C) se a replicação não ocorrer, não há nenhum prejuízo para o sistema.

D) a integração do *sharding* com a replicação promove a alta disponibilidade.

E) a utilização do *sharding* torna o processo de replicação desnecessário.

3) O Apache Cassandra utiliza uma estrutura de *hash* consistente para particionar os dados. Quanto a essa estrutura, escolha a alternativa correta:

A) É organizado em uma estrutura visualizada como um cubo.

B) Acaba gerando alta movimentação de dados.

C) É organizado em uma estrutura visualizada como uma estrela.

D) No particionamento do Cassandra, o número de *tokens* é limitado a 100.

E) É organizado em uma estrutura visualizada como um anel.

4) **Para realizar o *sharding*, o MongoDB necessita de uma estrutura mínima com três componentes: *shards*, roteadores de consulta e servidores de configuração. Quanto a esses componentes, é verdade que:**

A) os roteadores de consulta são a interface entre o aplicativo cliente e os *shards* propriamente ditos.

B) os roteadores de consulta utilizam informações fornecidas pelos próprios *shards*.

C) os servidores de configuração armazenam os dados e metadados de forma conjunta.

D) os *shards* armazenam metadados dos dados que estão nos servidores.

E) os *shards* armazenam unicamente o dado bruto não particionado.

5) **Os roteadores de consulta do MongoDB são chamados de *mongos*. Quanto aos mongos, é correto afirmar:**

A) Cada *shard* tem um *mongo* específico, não sendo possível buscar dados em diferentes *shards* a partir da mesma consulta.

B) É impossível saber a localização de um dado por meio dos metadados de cada partição.

C) A cada consulta é necessário que os mongos acessem os metadados, mesmo que essa busca seja repetida.

D) Os mongos apenas realizam as consultas do aplicativo cliente no banco de dados.

E) Se o resultado da consulta estiver em diferentes *shards*, os resultados aparecem de forma separada.

Saiba mais: Frameworks que utilizam sharding como forma de distribuição

Para ampliar o seu conhecimento a respeito desse assunto, veja abaixo as sugestões do professor:

Diferenças entre *sharding* e replicação

Nesta Unidade, você aprendeu os conceitos de *sharding* e replicação e algumas características de cada um deles. Confira, neste vídeo, quais são as principais diferenças entre eles e também como se diferenciam do *clustering*.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Cassandra – um sistema de armazenamento NoSQL altamente escalável

O Apache Cassandra é um sistema de armazenamento distribuído NoSQL muito utilizado por empresas mundo afora. Confira, neste artigo, uma breve explicação da sua arquitetura e conheça alguns estudos de caso em famosas redes sociais.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Sistemas de Gerenciamento de Bancos de Dados (3a Edição)

Nesta unidade, você aprendeu que o *sharding* pode ser realizado a partir da estrutura de *hash*. Veja, no livro *Sistemas de gerenciamento de banco de dados-3*, mais informações sobre índices baseados em *hash*, seus tipos e utilizações.

Conteúdo interativo disponível na plataforma de ensino!